UNIVERSITÉ DE PAU ET DES PAYS DE L'ADOUR
COLLÈGE STEE
SCIENCES ET TECHNOLOGIES POUR L'ÉNERGIE ET L'ENVIRONNEMENT



Java EE™

Enterprise Edition

Copyright AEN-Intech 2020-2022



- Pour pouvoir faire du JEE, il est nécessaire d'avoir un serveur d'applications Web.
- Tomcat a l'avantage d'être gratuit, libre, multi plateformes et très diffusé en entreprise. Il est composé :



  - D'un serveur HTTP frontal Apache chargé de recevoir les requêtes du navigateur (le même qui est utilisé pour faire du php, par exemple) ;
  - D'un serveur applicatif Tomcat chargé de faire tourner le code Java.

Pour plus d'informations, voir le cours « Tomcat.pdf »

Copyright AEN-Intech 2020-2022

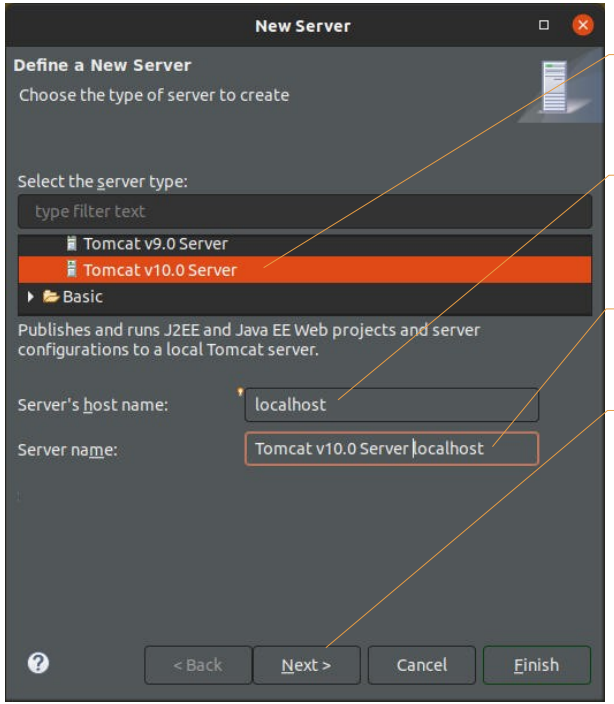
- La dernière version au moment d'écrire ce document est la version 10. vous la trouverez sur la page suivante : https://tomcat.apache.org/download-10.cgi
- Choisissez le « core » qui correspond à votre système d'exploitation (ne pas prendre l'installeur).



- Comme pour Eclipse, décompressez le zip dans un dossier dont le chemin ne contient pas d'espaces ou de caractères spéciaux et, de préférence, sur un SSD.

Copyright AEN-Intech 2020-2022

- Il y a plusieurs façon d'ajouter un serveur tomcat à Eclipse. On peut, par exemple, passer par le menu File → New → Server



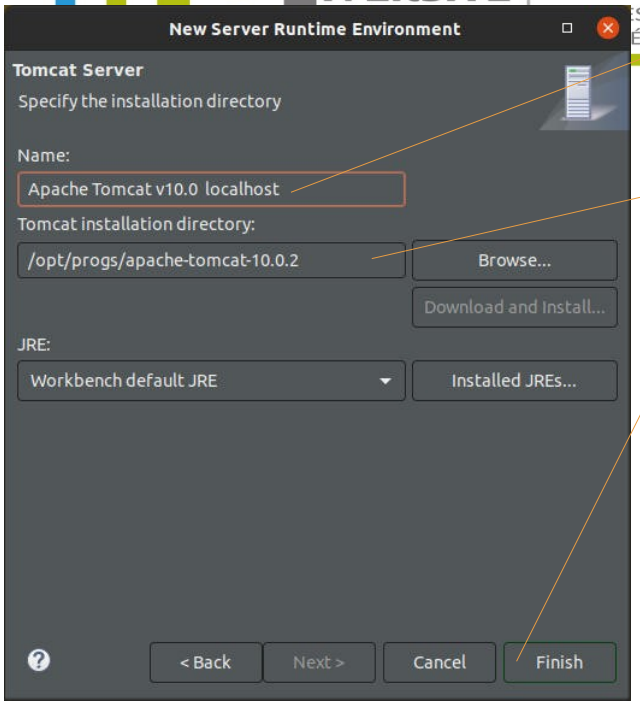
Sélectionnez la version de Tomcat adéquate.

Ce nom est celui qui va être utilisé dans les URLs.
Ne pas le modifier

Ce nom est interne à Eclipse, vous pouvez le modifier

Cliquez sur « Next »

Copyright AEN-Intech 2020-2022



Vous pouvez changer ce nom.

Renseignez, en le rentrant au clavier ou grâce au bouton « Browse… », le chemin d'installation de Tomcat.

Cliquez sur « Finish »

- Notez l'apparition d'un projet « Servers » dont chaque dossier contient les fichiers de paramétrage des différents serveurs configurés.

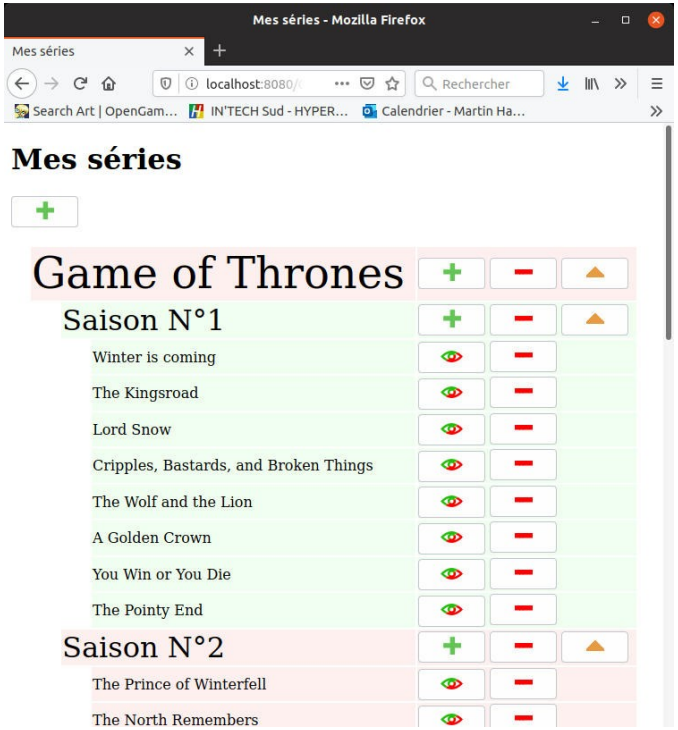
Copyright AEN-Intech 2020-2022

- Maintenant qu'Eclipse est correctement configuré, voyons à quoi ressemble un projet JEE.
- Importez le projet « **`GestionSeries`** » contenu dans le zip « **`GestionSeries.zip`** ». Pour cela,
  - dans la vue « **`Project explorer`** », faites un clic droit puis « **`Import`** ». Dans la fenêtre qui s'affiche, trouver le choix « **`Existing Projects into Workspace`** » puis « **`Next`** ».
  - Cochez le deuxième radio « **`Select archive file`** » et allez pointer sur le zip en question grâce au bouton « **`Browse`** » puis cliquez sur «**`Finish`** »

Note : Eclipse détecte une erreur à la première ligne de la jsp. C'est un bug Eclipse qui n'empêche en rien de faire tourner le projet et devrait être corrigé rapidement.

Copyright AEN-Intech 2020-2022

- Pour lancer le projet, faites un clic droit dessus puis « **Run As → Run on Server** ». Sélectionnez le serveur Tomcat précédemment installé et… magie !



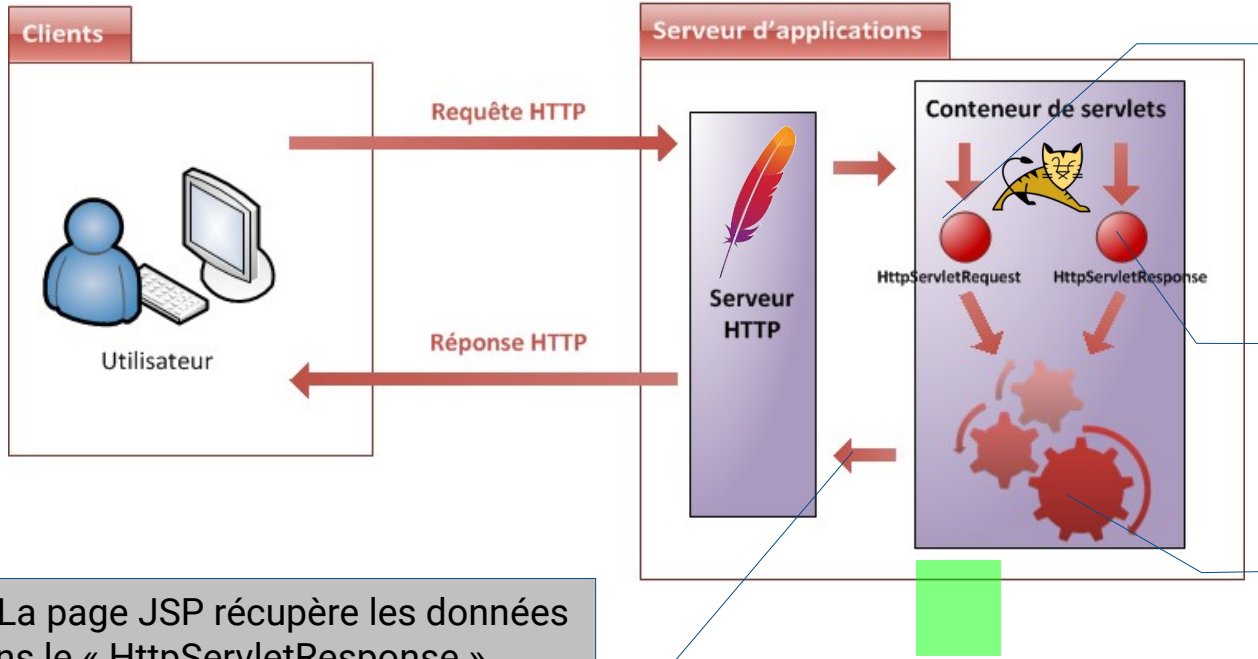
- En regardant les sources et en jouant avec l'interface web, essayez d'analyser le projet et proposez des pistes pour comprendre son fonctionnement.
- D'où viennent les données, comment arrivent-elles au navigateur, qui répond aux clics sur les boutons, … ?

Copyright AEN-Intech 2020-2022

- Pourquoi des dossiers séparés pour la partie HTTP et la partie Java ?
- Contrairement à des technologies comme le php où le code statique HTML et la partie dynamiquement créée par le serveur sont mélangés, l'architecture JEE sépare la partie applicative (en Java) de la partie présentation (en jsp et HTML statique).
- Cela permet plus de souplesse dans l'architecture (partie applicative et partie présentation sur deux serveurs différents, par exemple).
- On peut aussi ainsi créer des serveurs sans partie présentation (API par exemple).

Copyright AEN-Intech 2020-2022

Process d'une requête HTTP en contexte JEE /JSP.



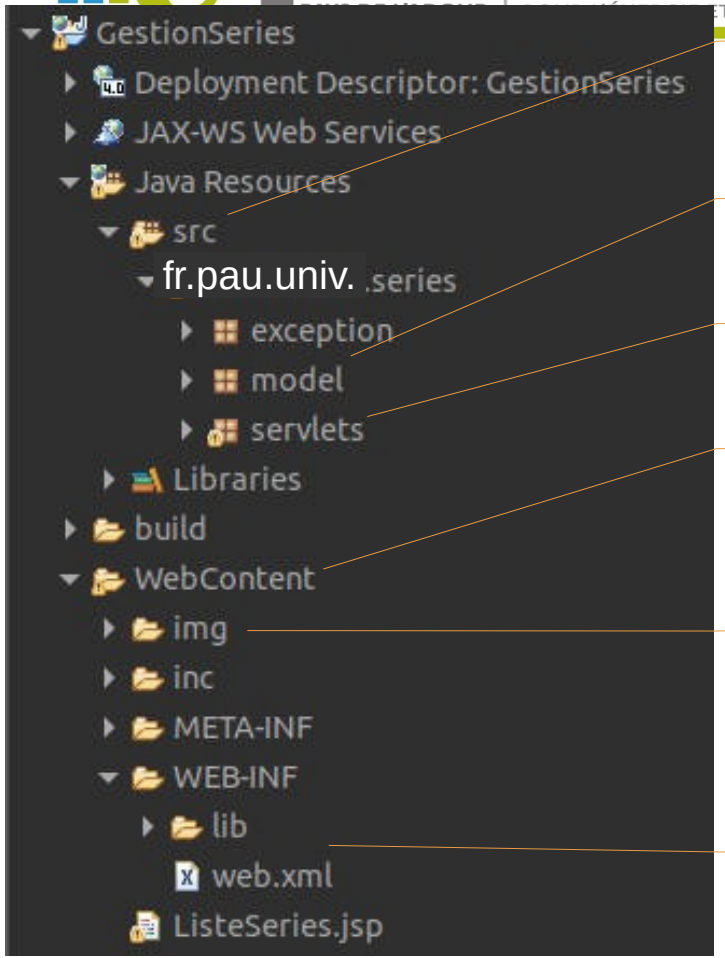
1) La requête du client est transmise par le serveur HTTP (Apache) au conteneur de servlets (Tomcat) qui l'encapsule dans un objet « HttpServletRequest » pour la mettre à disposition du servlet cible.

2) Le conteneur de servlets génère aussi un objet « HttpServletResponse » qui permettra au servlet cible de renvoyer les données.

3) Le servlet lit les paramètres contenus dans « HttpServletRequest » pour remplir le « HttpServletResponse » avec les données demandées.

4) La page JSP récupère les données dans le « HttpServletResponse » pour les mettre au formalisme HTTP.

Copyright AEN-Intech 2020-2022



Le dossier « **src** » contient toutes les classes Java responsables du back-office. Elles sont chargées de l'accès aux données et de l'application des règles métier.

Le package « **model** » contient toutes les classes du modèle de données.

Le package « **servlet** » contient tous les servlets. Ce sont des classes particulières chargées de répondre aux requêtes HTTP.

Le dossier « **WebContent** » est une spécificité Eclipse. C'est lui qui contient ce qui, après déploiement, sera à la racine du site...

… on y trouve donc toute la partie « front-end » de l'application s'il y en a une (images, css, jss, etc + pages jsp).
Nous n'insisterons pas sur cette partie front en JSP, cette option étant de moins en moins utilisée au bénéfice de fronts plus évolués type Angular, Vue.js, React.js, etc.

Le fichier web.xml est celui qui paramètre le serveur. Il est, pour l'instant relativement simple mais nous verrons par la suite ce qu'il peut contenir.

Copyright AEN-Intech 2020-2022

- Un servlet est une classe particulière ayant les éléments suivants :

Un servlet doit répondre à une requête HTTP particulière. Celle-ci sera déterminée soit par une annotation **`@WebServlet`** comme ici ou dans le web.xml (voir plus loin)

Un servlet étend la classe « **`HttpServlet`** »

```
@WebServlet(urlPatterns = "/liste")
public class ListeSeries extends HttpServlet {

    @Override
    public void doGet(final HttpServletRequest request, final HttpServletResponse response)
            throws ServletException, IOException {
        request.setAttribute("series", DataProvider.getInstance().getAllSeries());
        getServletContext().getRequestDispatcher("/ListeSeries.jsp").forward(request, response);
    }
}
```

La classe « **`HttpServlet`** » possède une méthode pour répondre à chacun des types de requêtes HTTP (GET, POST, PUT, DELETE, …).
Par défaut, ces méthodes renvoient un statut 405 (METHOD_NOT_ALLOWED). Il suffit de surcharger les méthodes qui nous intéressent pour que le serveur réponde positivement aux requêtes.

Copyright AEN-Intech 2020-2022

- L'annotation « `@WebServlet` » est un raccourci qui remplace les attributs suivants dans le fichier « web.xml » :

- Association nom / classe :

```
<servlet>
    <servlet-name>ListeSeries</servlet-name>
    <servlet-class>com.intech.series.servlets.ListeSeries</servlet-class>
</servlet>
```

Nom Logique du servlet (peut prendre n'importe quelle valeur pourvu que ce soit la même que celle du tag « `servlet-mapping` »)

Classpath du servlet

- Association nom / URL :

```
  <servlet-mapping>
    <servlet-name>ListeSeries</servlet-name>
    <url-pattern>/liste</url-pattern>
  </servlet-mapping>
```

Nom Logique du servlet
(le même que dans le tag « servlet »)

Url associée.

Copyright AEN-Intech 2020-2022

- En résumé, cet extrait de classe java ...

```
@WebServlet(urlPatterns = "/liste/")
public class ListeSeries extends HttpServlet {
```

- est équivalent à ces tags dans le fichier web.xml.

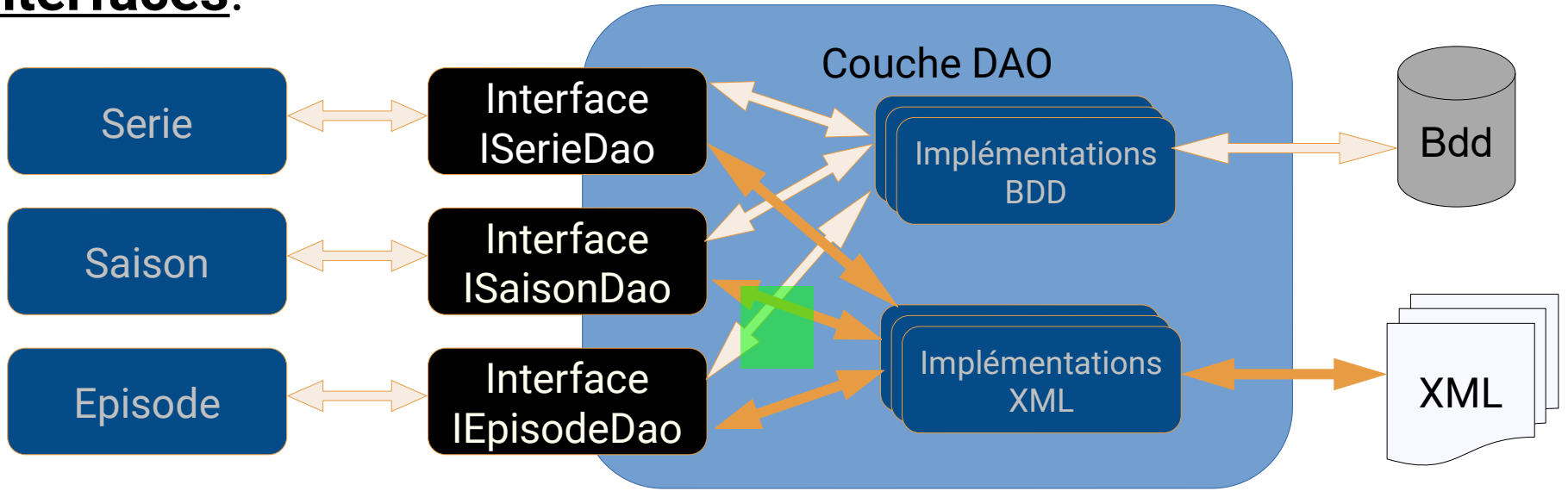
```
<servlet>
    <servlet-name>ListeSeries</servlet-name>
    <servlet-class>fr.pau.univ.series.servlets.ListeSeries</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ListeSeries</servlet-name>
    <url-pattern>/liste/</url-pattern>
</servlet-mapping>
```

- Ils servent tous les deux à associer une partie d'URL (ex : « `http://localhost:8080/GestionSeries/liste/` ») à un servlet spécifique.

La première partie de l'URL provient du tag « `display-name` » du fichier web.xml

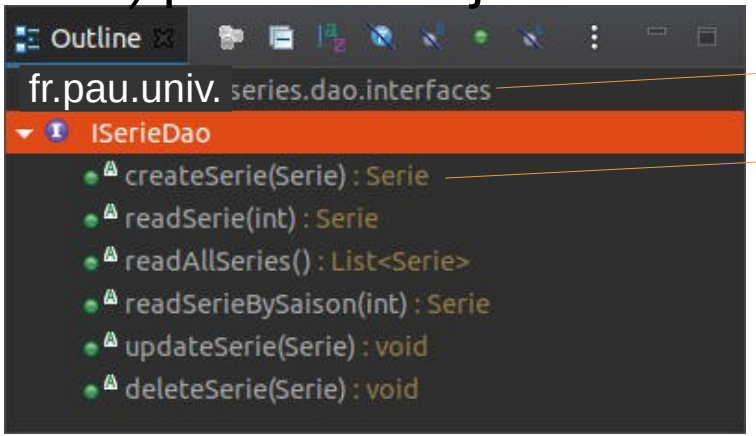
Copyright AEN-Intech 2020-2022

- Pour pouvoir stocker nos séries, nous allons définir une couche supplémentaire : la couche DAO (Data Accès Objets).
- Il s'agit d'une couche se plaçant entre nos objets métier et le stockage afin de couper tout lien fort entre eux par le biais d'**interfaces**.



Copyright AEN-Intech 2020-2022

- Les interfaces DAO vont définir les méthodes CRUD (Create, Read, Update et Delete) pour nos objets métier. Voici un exemple pour les Séries.



Notez le nom du package

La méthode de création renvoie l'objet créé

Attention : une interface étant un contrat de service, il est **IMPÉRATIF** de renseigner la Javadoc pour toutes ses méthodes

- Définissez les interfaces DAO pour les autres objet en suivant le même modèle CRUD.
- Nous allons nous contenter d'une sauvegarde en BDD pour l'instant. Codez, dans un autre package (nommé «**fr.pau.univ.series.dao.impl.bdd** »), leurs implémentations BDD avec comme nom, par exemple « **`SerieDaoBdd`** ». Laissez les méthodes vides pour le moment.

Copyright AEN-Intech 2020-2022

- Nous avons donc défini des interfaces afin de casser une dépendance forte entre nos objets métiers et leur stockage et commencé à coder les implémentations BDD de ces interfaces. Il va maintenant falloir utiliser ces évolutions de notre projet.

- Renommez la classe « **`DataProvider`** » en « **`DaoFactory`** » et déplacez-la dans le package « **`fr.pau.univ.series.dao`** ».

- C'est le moment de découvrir (ou revoir) les design patterns « **`Singleton`** » et « **`Factory`** ». 

Copyright AEN-Intech 2020-2022

- Un servlet est une classe particulière ayant les éléments suivants :

Un servlet doit répondre à une requête HTTP particulière. Celle-ci sera déterminée soit par une annotation **`@WebServlet`** comme ici ou dans le web.xml (voir plus loin)

Un servlet étend la classe « **`HttpServlet`** »

```
@WebServlet(urlPatterns = "/liste")
public class ListeSeries extends HttpServlet {

    @Override
    public void doGet(final HttpServletRequest request, final HttpServletResponse response)
            throws ServletException, IOException {
        request.setAttribute("series", DataProvider.getInstance().getAllSeries());
        getServletContext().getRequestDispatcher("/ListeSeries.jsp").forward(request, response);
    }
}
```

La classe « **`HttpServlet`** » possède une méthode pour répondre à chacun des types de requêtes HTTP (GET, POST, PUT, DELETE, …).
Par défaut, ces méthodes renvoient un statut 405 (METHOD_NOT_ALLOWED). Il suffit de surcharger les méthodes qui nous intéressent pour que le serveur réponde positivement aux requêtes.

Copyright AEN-Intech 2020-2022

- L'annotation « `@WebServlet` » est un raccourci qui remplace les attributs suivants dans le fichier « web.xml » :

- Association nom / classe :

```
<servlet>
    <servlet-name>ListeSeries</servlet-name>
    <servlet-class>fr.pau.univ.series.servlets.ListeSeries</servlet-
class>
</servlet>
```

Nom Logique du servlet (peut prendre n'importe quelle valeur pourvu que ce soit la même que celle du tag « `servlet-mapping` »)

Classpath du servlet

- Association nom / URL :

```
  <servlet-mapping>
    <servlet-name>ListeSeries</servlet-name>
    <url-pattern>/liste</url-pattern>
  </servlet-mapping>
```

Nom Logique du servlet
(le même que dans le tag « servlet »)

Url associée.

Copyright AEN-Intech 2020-2022

- En résumé, cet extrait de classe java ...

```
@WebServlet(urlPatterns = "/liste/")
public class ListeSeries extends HttpServlet {
```

- est équivalent à ces tags dans le fichier web.xml.

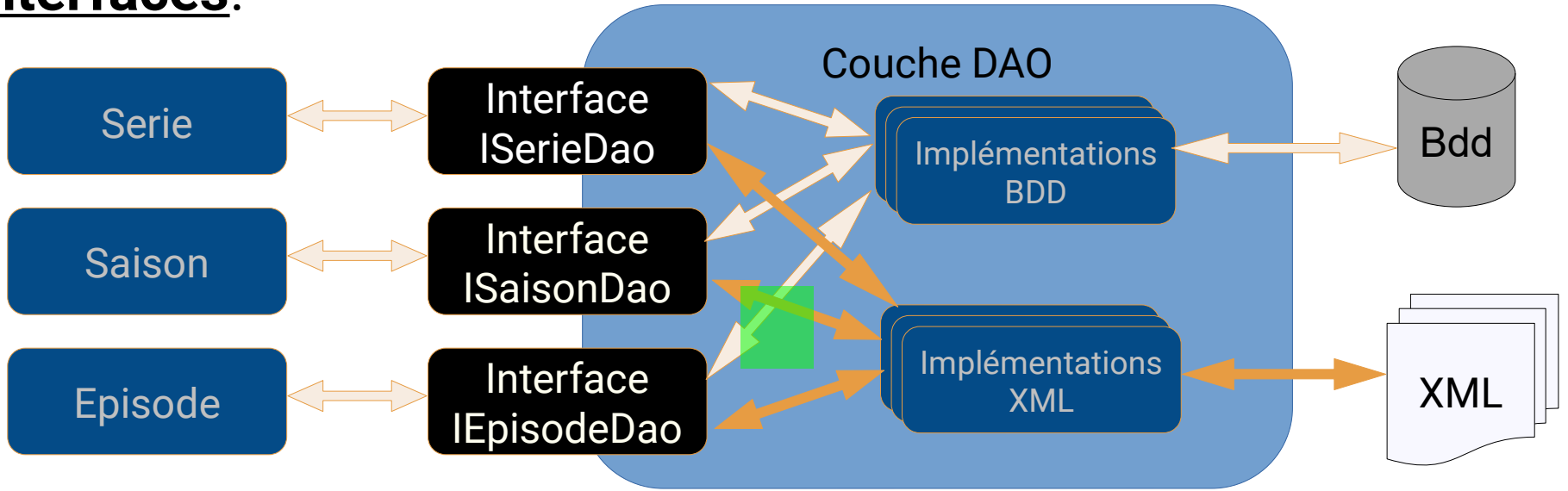
```
<servlet>
    <servlet-name>ListeSeries</servlet-name>
    <servlet-class>fr.pau.univ.series.servlets.ListeSeries</servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name>ListeSeries</servlet-name>
    <url-pattern>/liste/</url-pattern>
</servlet-mapping>
```

- Ils servent tous les deux à associer une partie d'URL (ex : « `http://localhost:8080/GestionSeries/liste/` ») à un servlet spécifique.

La première partie de l'URL provient du tag « `display-name` » du fichier web.xml

Copyright AEN-Intech 2020-2022

- Pour pouvoir stocker nos séries, nous allons définir une couche supplémentaire : la couche DAO (Data Accès Objets).
- Il s'agit d'une couche se plaçant entre nos objets métier et le stockage afin de couper tout lien fort entre eux par le biais d'**interfaces**.



Copyright AEN-Intech 2020-2022

- Notre **`DaoFactory`** reste un singleton (que l'on reconnaît à **3 facteurs**) :

```
private static DaoFactory instance;

public static final DaoFactory getInstance() throws DaoException {
     if (instance == null) {
          instance = new DaoFactory();
     }
     return instance;
}

private DaoFactory() throws DaoException {
}
```

Une variable du type de notre classe définie en tant que variable **privée et statique**.
Celle-ci est en général nommée « `instance` » pour plus de clarté, mais ce n'est pas obligatoire.

Une méthode **publique et statique** nommée « `getInstance` » qui renvoie la seul instance du singleton après l'avoir, si besoin, « lazy loadée ».

Un **constructeur privé** afin d'empêcher toute instanciation de la classe depuis l'extérieur de celle-ci.

Copyright AEN-Intech 2020-2022

- La **DaoFactory** devient aussi une factory chargée de fabriquer nos différents DAO :

```
private ISerieDao serieDao = null;
private ISaisonDao saisonDao = null;
private IEpisodeDao episodeDao = null;

public ISerieDao getSerieDao() {
     if (this.serieDao == null) {
          this.serieDao = new SerieDaoBdd();
     }
     return this.serieDao;
}

public ISaisonDao getSaisonDao() {
     if (this.saisonDao == null) {
          this.saisonDao = new SaisonDaoBdd();
     }
     return this.saisonDao;
}

public IEpisodeDao getEpisodeDao() {
     if (this.episodeDao == null) {
          this.episodeDao = new EpisodeDaoBdd();
     }
     return this.episodeDao;
}
```

Les différents Dao sont stockés sous la forme d'attributs d'instance privés. **Ils sont typés par l'interface**, pas par une implémentation.

Des getters permettent de récupérer chaque type de Dao après un lazy loading éventuel. Là encore, **le type renvoyé par le getter est l'interface**, pas une implémentation.

En cas de changement de stockage, les seules lignes impactées **dans tout le projet** seront celles, Regroupées dans la factory, où on instancie les différents implémenteurs.

Copyright AEN-Intech 2020-2022

- Vous pouvez enlever tout le reste de l'ancienne classe **`DataProvider`** même si cela provoque des erreurs dans vos servlets. Nous allons les corriger.
- Voici, par exemple, la nouvelle version de la méthode **`doGet`** du servlet **`ListeSeries`** :

```
public void doGet(final HttpServletRequest request, final HttpServletResponse response)
          throws ServletException, IOException {
     try {
          request.setAttribute("series", DaoFactory.getInstance().getSerieDao().readAllSeries());
     } catch (DaoException e) {
          request.setAttribute("erreur", e.getMessage());
          e.printStackTrace();
     }
     this.getServletContext().getRequestDispatcher("/ListeSeries.jsp").forward(request, response);
}
```

Copyright AEN-Intech 2020-2022

- Normalement, vous avez déjà installé MySQL sur vos PC lors des semestres précédents. Nous allons utiliser ce moteur de BDD.
- Ouvrez votre gestionnaire de BDD et créez un schéma « GestionSeries » vierge.
- Pour pouvoir se connecter à une base de données depuis Java, il va nous falloir un « Driver JDBC » (JDBC pour Java Data Base Conectivity).
- Ce driver dépendant du type de la base (MySQL, Oracle, …) et de la version de son moteur, nous allons utiliser un gestionnaire de projet pour gérer les dépendances.

Copyright AEN-Intech 2020-2022

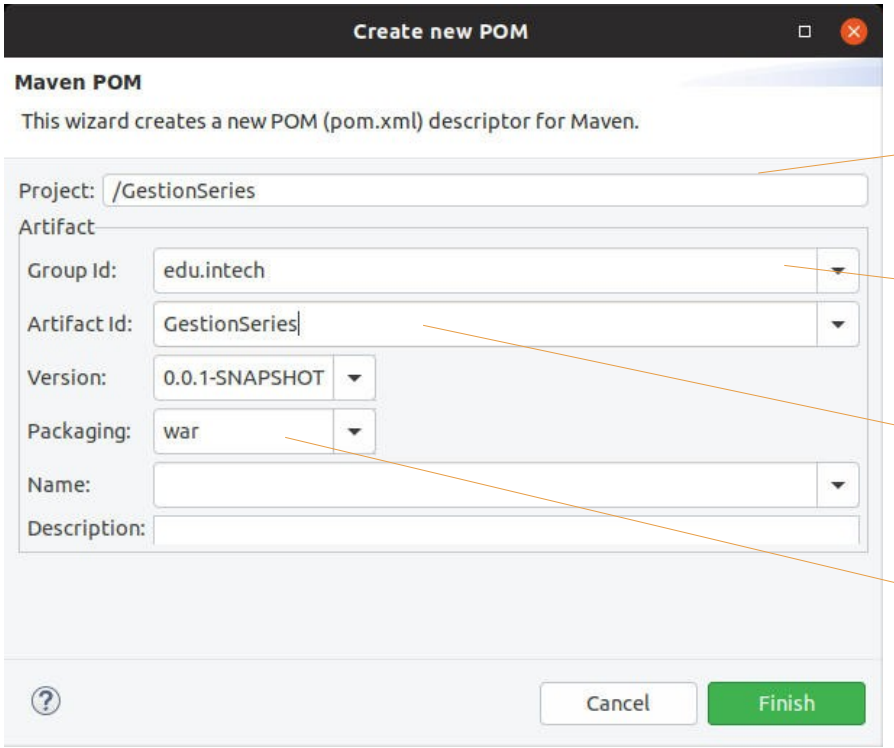
UNIVERSITÉ DE PAU ET DES PAYS DE L'ADOUR | COLLÈGE STEE SCIENCES ET TECHNOLOGIES POUR L'ÉNERGIE ET L'ENVIRONNEMENT

Il est temps de faire une petite parenthèse...



Copyright AEN-Intech 2020-2022

- Menu contextuel projet « Configure → Convert to Maven project »



Le nom du projet n'est pas modifiable.
Il s'agit du nom du projet Eclipse à « maveniser »

Le group id est très important. Il permettra de différencier différents projets sur le même Sujet en les préfixant par leurs éditeurs.

L'artefact Id est l'identifiant du projet dans le groupe.

Nous développons une application web : le packaging est donc « war ».

Copyright AEN-Intech 2020-2022

- Le fait de « maveniser » le projet va faire quelques modifications à la structure de celui-ci (apparition d'un dossier « target », entre autres). Mais le changement le plus important est la création du fichier « pom.xml » à la racine du projet.
- Via le site « https://mvnrepository.com/ », récupérez la dernière version de l'artefact correspondant à votre base de données pour la rajouter dans le pom.xml. Attention, il faut ajouter un tag « **dependencies** » sous le tag « **build** » pour encapsuler les différents tags « dependency ».
- On en profitera pour enlever les jar dans WEB-INF/lib pour les remplacer par la dépendance :

```
<dependency>
<groupId>org.glassfish.web</groupId>
<artifactId>jakarta.servlet.jsp.jstl</artifactId>
<version>2.0.0</version>
</dependency>
```

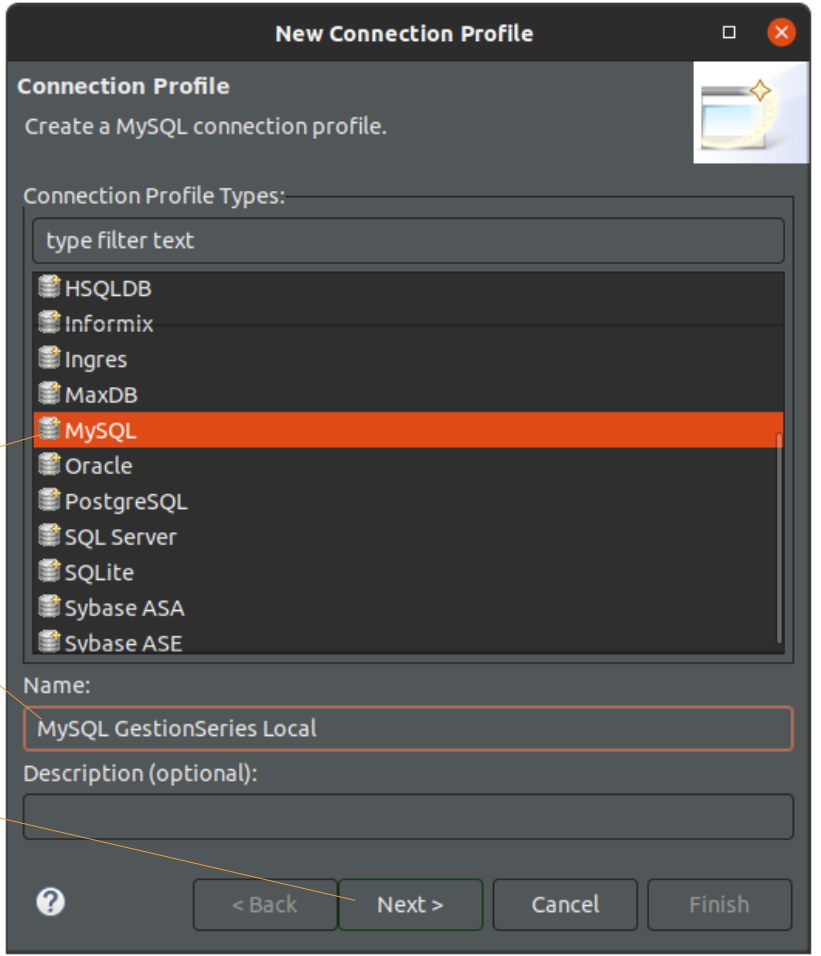
Copyright AEN-Intech 2020-2022

- L'étape suivante consiste à lier la base de donnée créée avec Eclipse. Ouvrez la vue « Data Source Explorer » pour créer une nouvelle « Database connection ».

Le profil sera, bien entendu, de type votre base de données Exemple « MySQL » ; « PostgreSQL », etc.

Eclipse gère une connexion par schema de base. Donnez donc à celle-ci un nom explicite avec le nom du schéma et le fait qu'il soit local.

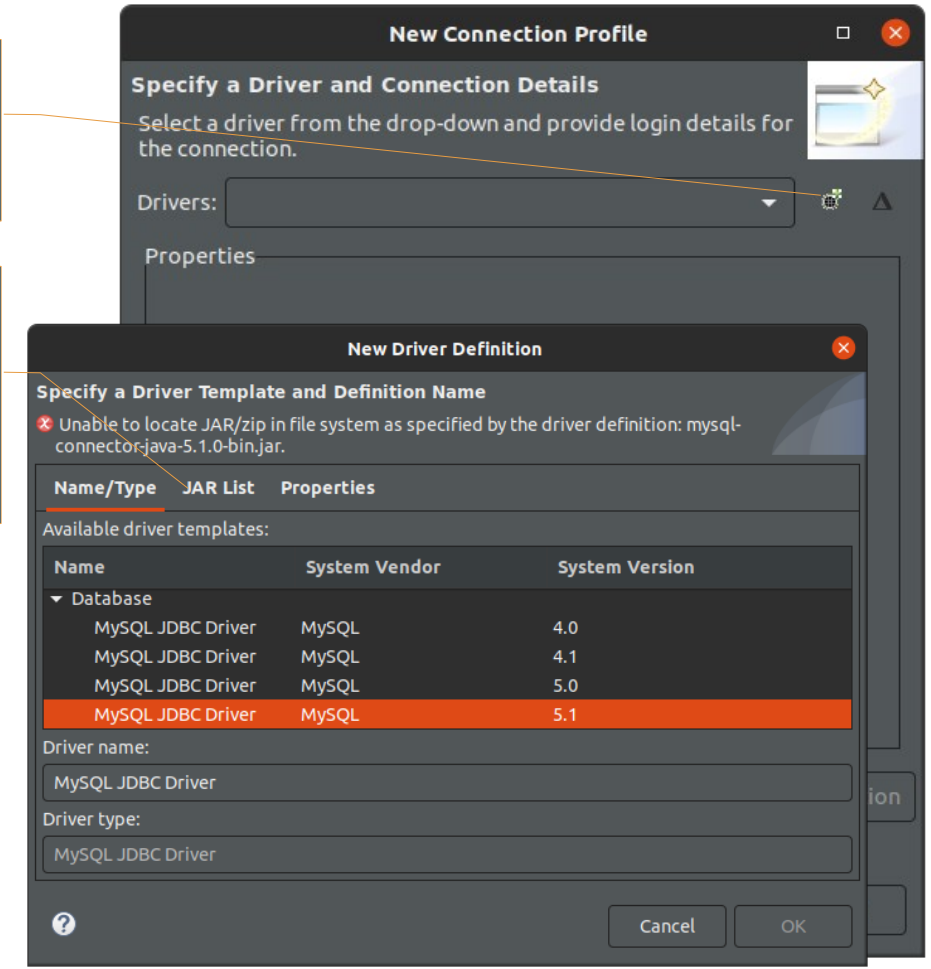
Next...



Copyright AEN-Intech 2020-2022

Nous allons devoir définir un nouveau driver.
Ceci ne sera plus à faire pour les prochaines connexions.

La nouvelle fenêtre qui s'ouvre nous permet de choisir la version de driver à utiliser. Sélectionnez la dernière version (5.1). Cela affiche un message d'erreur en haut de fenêtre. Pour le corriger, sélectionnez l'onglet « JAR list ».



Copyright AEN-Intech 2020-2022

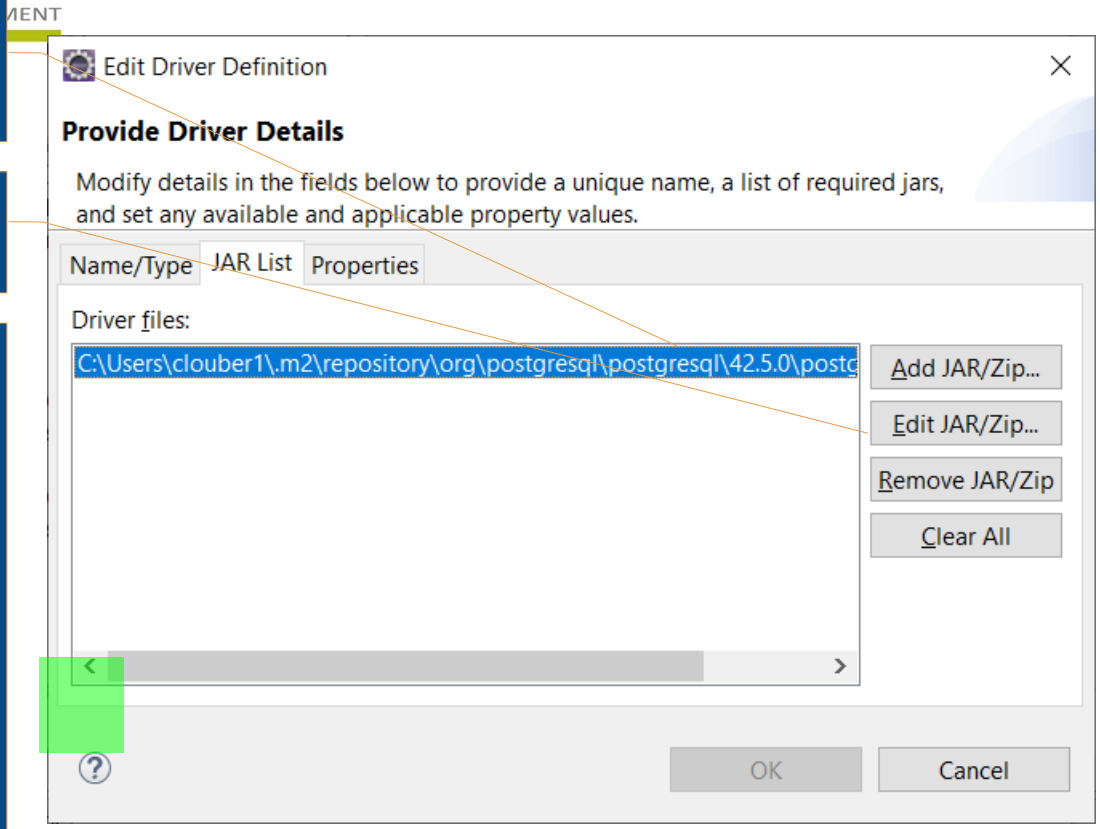
Pour une raison inconnue, Eclipse est incapable de trouver le jar qui sert de pilote.
Sélectionnez la ligne qui l'affiche...

... puis cliquez sur le bouton « Edit JAR / zip »

Cela va ouvrir une fenêtre de recherche de fichiers.

Allons chercher le pilote qui a été ramené par Maven lorsque nous avons rajouté la dépendance « org.postgresql » à notre projet.

Allez trouver, dans votre dossier utilisateur, le répertoire « **.m2** ». Dans celui-ci, trouvez le jar « **`mysql-connector-java-x.y.z.jar`** » situé dans le sous dossier « **`repository/org/postgresql/postgresql/x.y.z`** »
Le message d'erreur en haut de la fenêtre va disparaître et le bouton « OK » s'activer. Cliquez-le.



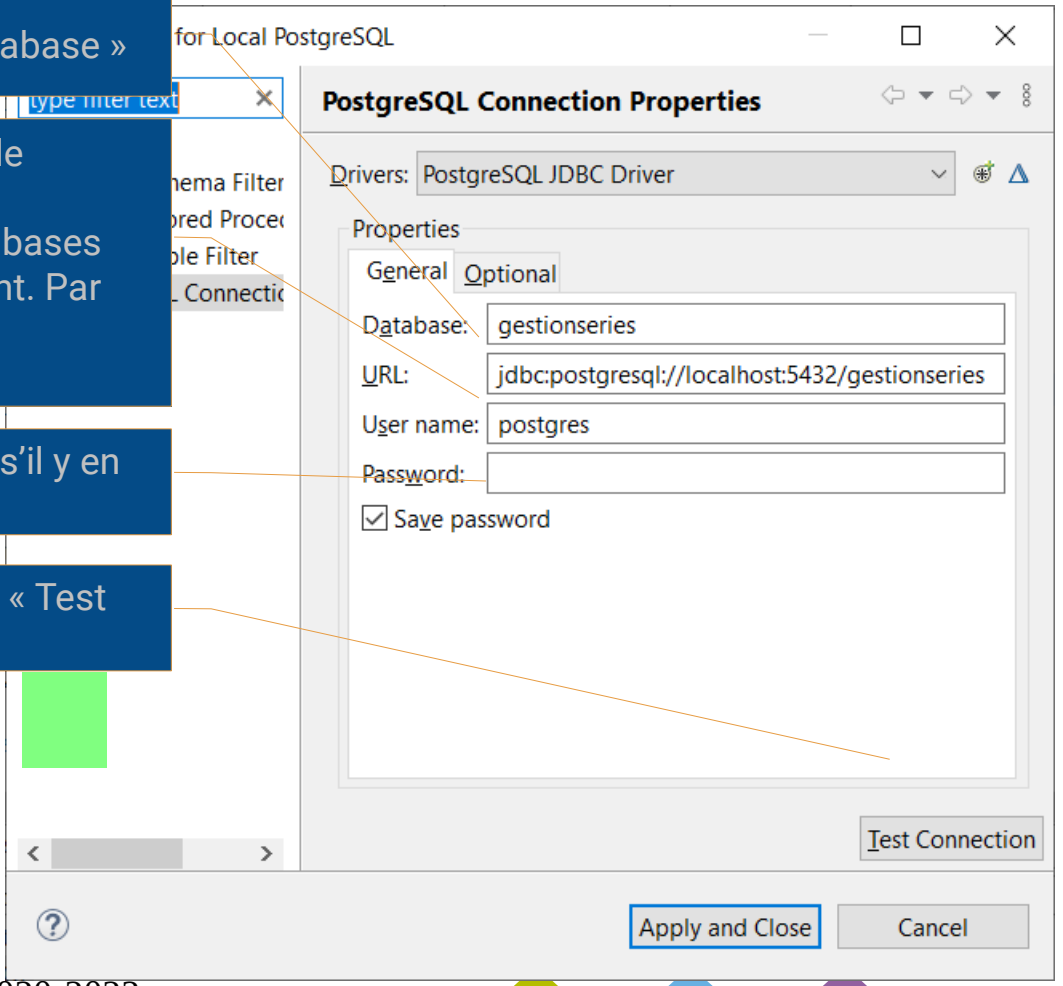
Copyright AEN-Intech 2020-2022

Renseignez le nom du schéma dans le champ « Database »

Vérifiez le port et modifiez le nom du schéma dans le champ « URL ».
Note : l'utilisation d'une URL permet d'atteindre des bases distantes lorsque le moteur de celles-ci le permettent. Par exemple, pour MySQL, l'URL est de type:
`jdbc:mysql://nomhote:port/nombdd`

Renseignez le nom d'utilisateur et le mot de passe (s'il y en a un, cochez aussi la case « Save Password »).

Avant de cliquer sur « Finish », vérifiez qu'un clic sur « Test Connexion » vous répond bien « Ping Successfull »



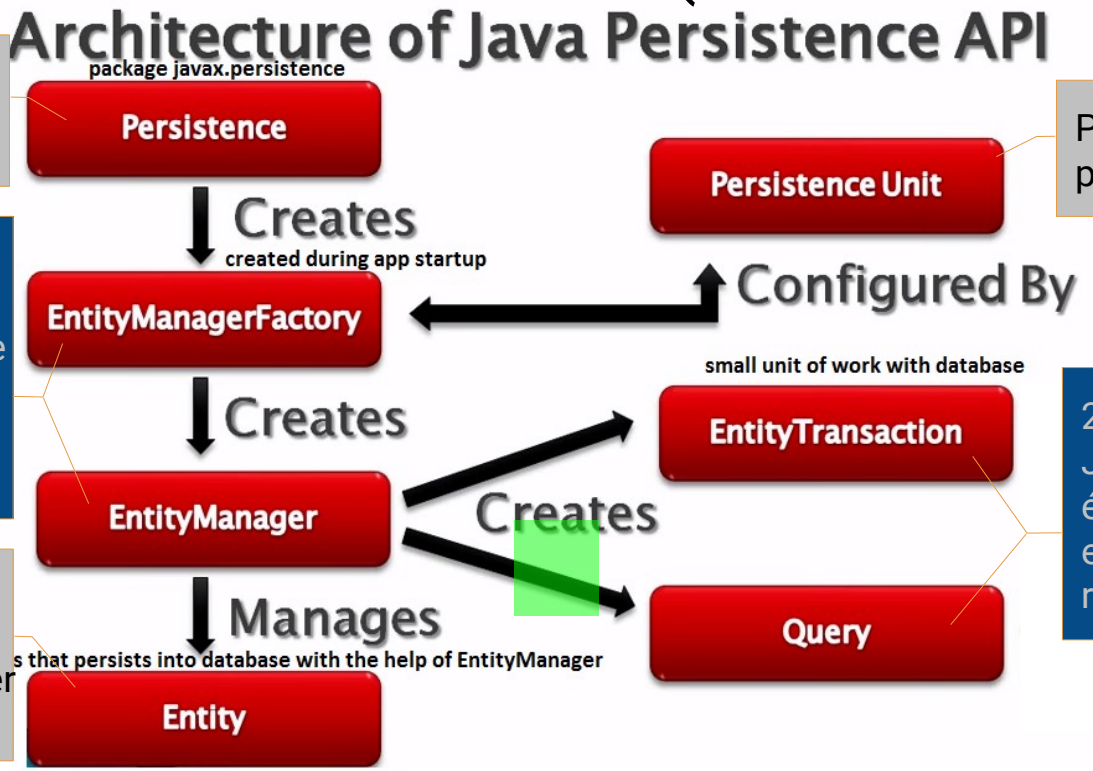
Copyright AEN-Intech 2020-2022

- Afin d'automatiser la persistance de nos données, nous allons utiliser le framework JPA (Java Persistence API).

Ajout d'une implémentation JPA au projet.

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.

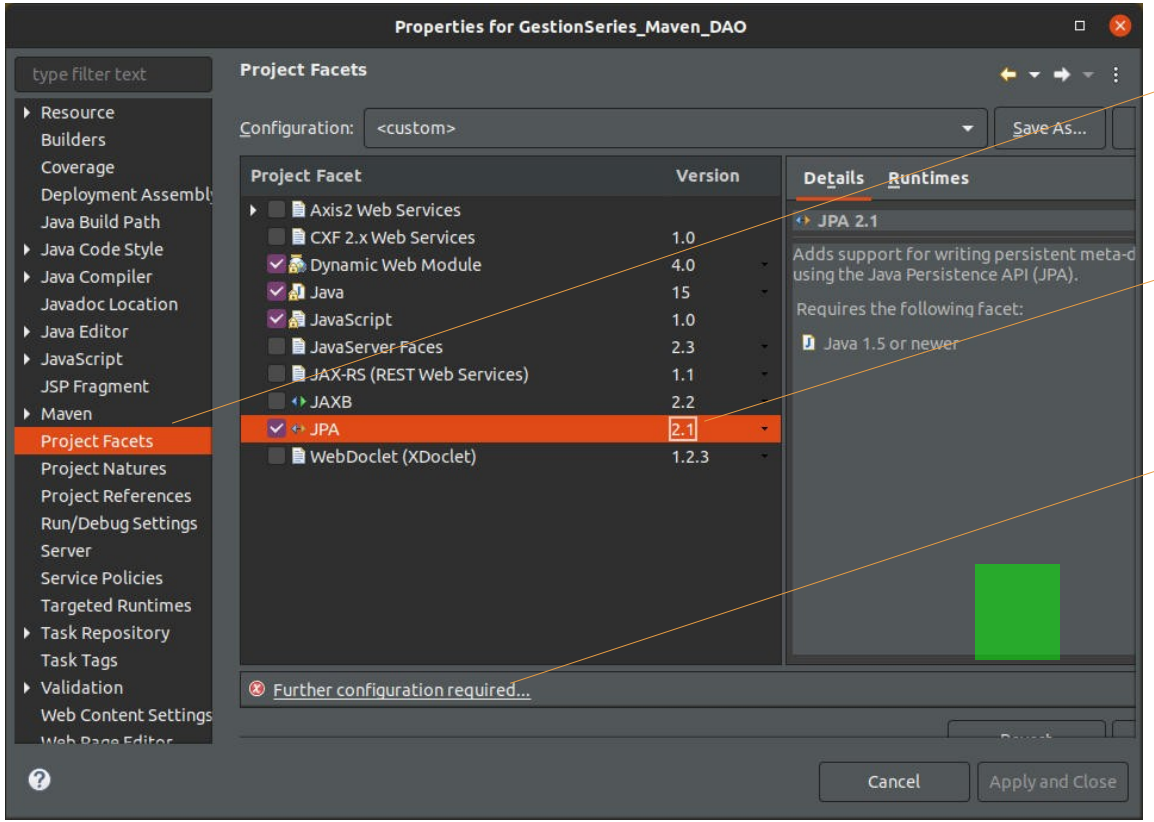


Paramétrage via le fichier persistence.xml

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Copyright AEN-Intech 2020-2022

- Afin d'utiliser au mieux l'intégration de JPA à Eclipse, ajoutons la facette JPA à notre projet par l'intermédiaire des propriétés du projet :

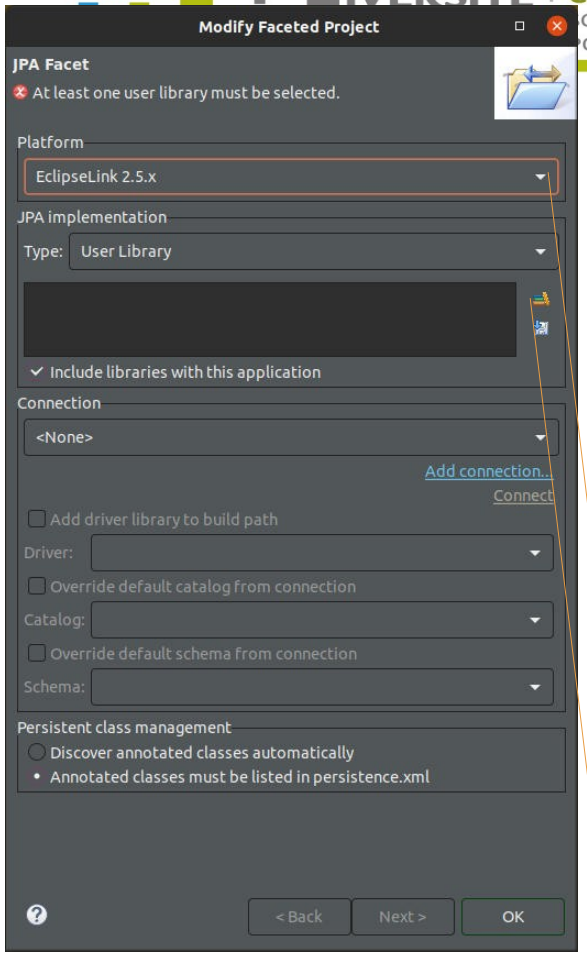


Project Facets

Cochez JPA et changez la version vers 2.1

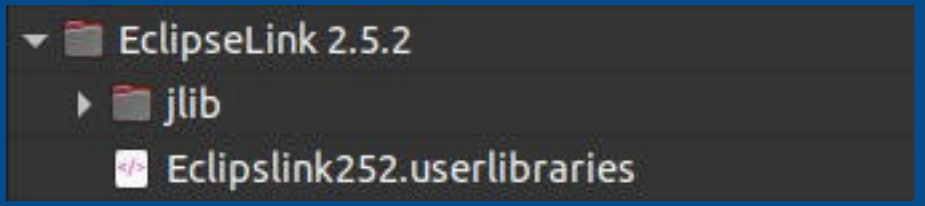
Cliquez sur le message d'erreur.

Copyright AEN-Intech 2020-2022



- JPA étant défini sous forme d'interfaces, il va falloir utiliser l'une de ses implémentations.

Récupérez l'archive « EclipseLink 2.5.2.zip » et décompactez-la directement à la racine de votre disque « C : » Vous devez obtenir l'arborescence suivante :

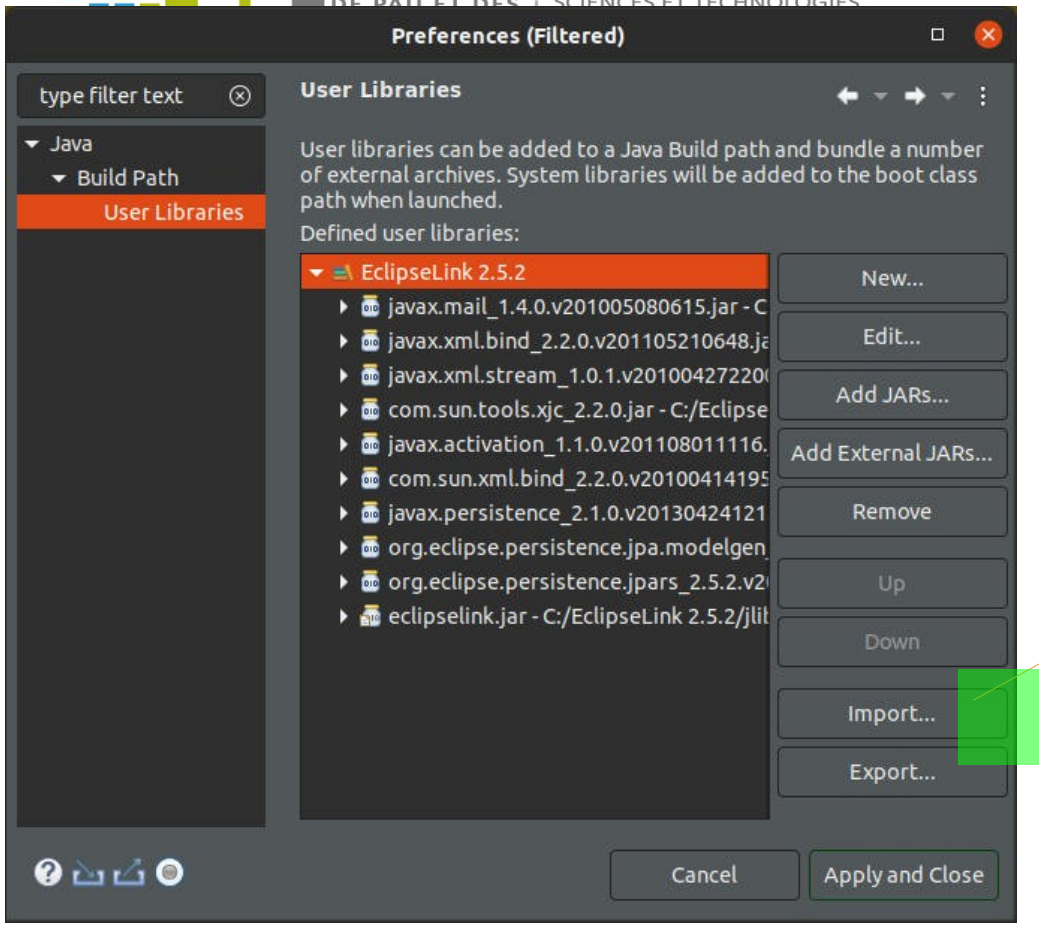


Note : si vous avez décompacté ailleurs qu'à la racine de « C: », il faudra ouvrir le fichier « Eclipslink252.userlibraries » avec un éditeur de texte et y modifier les chemins afin de refléter la position réelle des fichiers sur le disque.

Choisissez « EclipseLink 2.5.x » si disponible

Cliquez sur l'icone « Manage libraries »

Copyright AEN-Intech 2020-2022

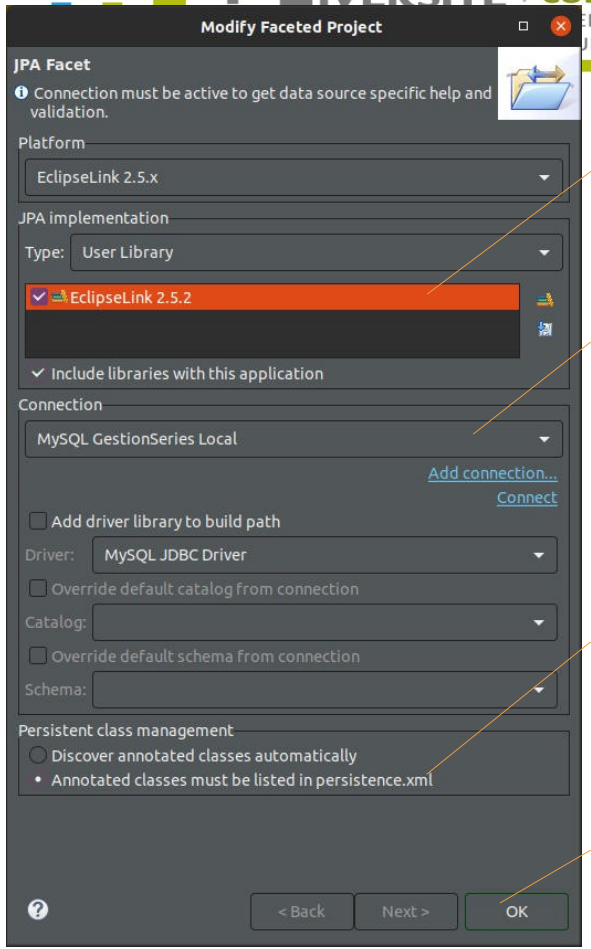


Cliquez sur le bouton « Import » et allez chercher le fichier « Eclipslink252.userlibraries » extrait de l'archive auparavant.

Vous devriez obtenir la liste de jars au centre.

Cliquez sur « Apply and Close »

Copyright AEN-Intech 2020-2022



Vous pouvez maintenant sélectionner la libraire « EclipseLink 2.5.2 ».

Dans la liste déroulante « Connection », sélectionnez la connexion vers la base de donnée préalablement paramétrée

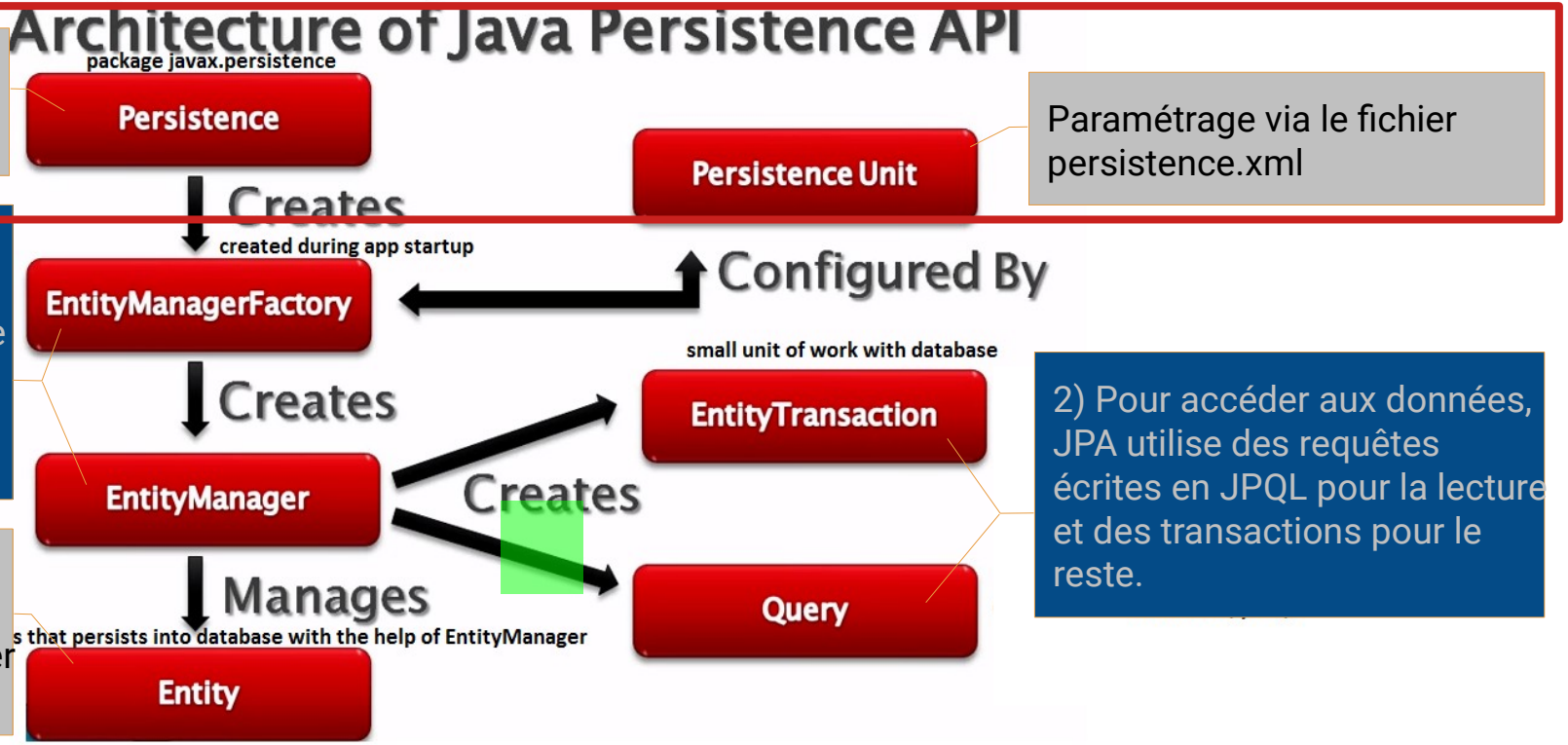
Pour des raisons d'optimisation et de maintenance, laissez le deuxième radio-bouton coché. Cela impose que les entités que va gérer JPA soient explicitement déclarées plutôt que dynamiquement découvertes.

Cliquez sur « OK »

Copyright AEN-Intech 2020-2022

- La principale modification du projet est l'apparition d'un fichier persistence.xml. Dans notre schéma d'architecture, nous avons donc mis en place les éléments suivants :

Ajout d'une implémentation JPA au projet.



Paramétrage via le fichier persistence.xml

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.

Copyright AEN-Intech 2020-2022

- Le fichier persistence.xml a été généré, mais il est pour l'instant assez vide. Remplissons-le un peu :

```xml
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="2.1"
xmlns="http://xmlns.jcp.org/xml/ns/persistence"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd">
      <persistence-unit name="GestionSeries">
            <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
            <exclude-unlisted-classes>true</exclude-unlisted-classes>
            <properties>
                  <property name="javax.persistence.jdbc.driver" value="org.postgresql.Driver" />
                  <property name="javax.persistence.jdbc.url"
value="jdbc:postgresql://localhost:5432/gestionseries" />
                  <property name="javax.persistence.jdbc.user" value="root" />
                  <property name="javax.persistence.jdbc.password" value="monMotDePasse" />
                  <property name="eclipselink.logging.level" value="FINE" />
            </properties>
      </persistence-unit>
</persistence>
```

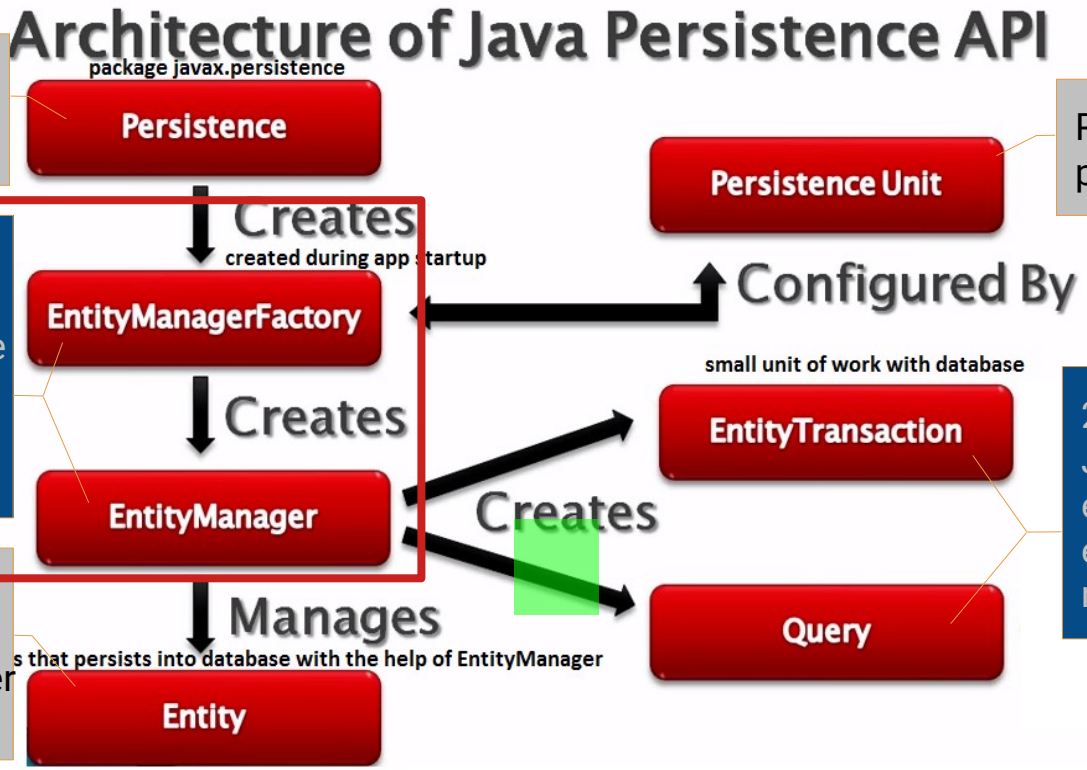
Comme dit précédemment, pour des raisons d'optimisation et de maintenance, les entités que va gérer JPA sont explicitement déclarées plutôt que dynamiquement découvertes.

Ce niveau de trace est à réserver au développement ou au débogage. En production, on le passera à « INFO » pour optimiser le serveur.

Copyright AEN-Intech 2020-2022

- Attaquons nous maintenant à la création et à la mise à disposition de l'EntityManager.

Ajout d'une implémentation JPA au projet.



Paramétrage via le fichier persistence.xml

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.

Copyright AEN-Intech 2020-2022

- L'EntityManager doit être initialisé « Au démarrage de l'application ou du serveur ». Dans une application Java normale, c'est relativement simple, il suffit de réaliser cela dans la méthode « main ».
- Comment faire dans un contexte de serveur Tomcat ?
- Il suffit de créer un implémenteur de l'interface « **`javax.servlet.ServletContextListener`** ». Celle-ci propose deux méthodes, l'une gère l'initialisation du serveur, l'autre son extinction.
- Nous allons donc initialiser l'EntityManager dans la méthode appelée lors du démarrage du serveur.

Copyright AEN-Intech 2020-2022

```java
@WebListener
public class SeriesContextListener implements ServletContextListener {

    private static EntityManager em;

    @Override
    public void contextInitialized(final ServletContextEvent sce) {
        final EntityManagerFactory emf = Persistence.createEntityManagerFactory("GestionSeries");
        em = emf.createEntityManager();
    }

    @Override
    public void contextDestroyed(final ServletContextEvent sce) {
        if (em != null) {
            em.getEntityManagerFactory().close();
        }
    }

    public static EntityManager getEntityManager() {
        return em;
    }
}
```

Annotation prévenant le serveur que cette classe est un web Listener

L'**EntityManager** est stocké dans une variable statique.

Elle est initialisée au lancement du serveur avec le nom défini dans **persistence.xml**

À l 'arrêt du serveur, on libère les ressources si nécessaire.

Par intermédiaire d'une méthode statique, on rend disponible l'**EntityManager** à tout le code serveur.

Copyright AEN-Intech 2020-2022

- Nous avons donc un EntityManager disponible au lancement du serveur. Où va t'on l'utiliser ?
- Afin de bien découper les couches, la DaoFactory ne doit pas s'occuper de la BDD. Nous allons donc créer un autre singleton dans le package `(…).dao.implementations.bdd`

```
private static DaoBddHelper instance;
private final EntityManager em;

private DaoBddHelper() throws DaoException {

    try {

        this.em = SeriesContextListener.getEntityManager();
        System.out.println("EntityManager créé.");
    } catch (final Exception e) {
        throw new DaoException("Impossible de créer l'EntityManager.", e);
    }
}
```

L'**EntityManager** est stockée dans une variable d'instance. Comme notre **DaoBddHelper** est un singleton, cela garanti son unicité.

On modifie le constructeur privé de la **DaoFactory** pour aller chercher l'**EntityManager**.

Notez la présence d'une Exception « custom » que l'on étend de l'Exception standard pour encapsuler toutes nos erreurs DAO.

Copyright AEN-Intech 2020-2022

- L'**`EntityManager`** sera disponible via un simple getter :

```
public EntityManager getEm() {
    return this.em;
}
```

- Et le **`DaoBddHelper`** sera stocké dans chaque implémentation des DAO :

```
public class SerieDaoBdd implements ISerieDao {

    private final DaoBddHelper bdd;

    public SerieDaoBdd() throws DaoException {
        this.bdd = DaoBddHelper.getInstance();
    }
// Suite de la classe ...
```

Le constructeur renvoie maintenant une **`DaoException`**.

- À dupliquer, bien sur sur **`SaisonDAOBdd`** et **`EpisodeDaoBdd`**

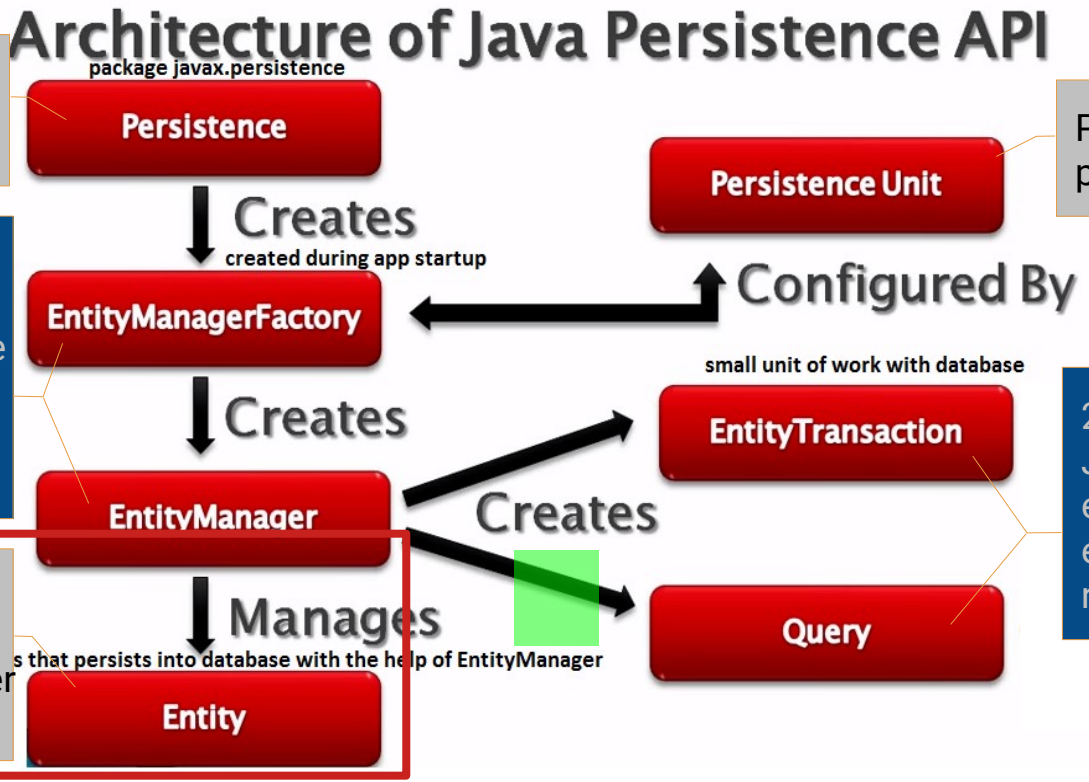
Copyright AEN-Intech 2020-2022

- Maintenant que l'EntityManager est disponible, voyons comment gérer les entités.

Ajout d'une implémentation JPA au projet.

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.



Paramétrage via le fichier persistence.xml

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Copyright AEN-Intech 2020-2022

- C’est le moment d’annoter nos classes dans le package « **`model`** ».
- Les annotations vont permettre à JPA de gérer la persistance de nos objets.
- Ceux sont elles qui déterminent :
  - La correspondance entre les objets de notre modèle métier et les tables dans la base de données.
  - Le mapping entre les attributs de nos objets métier et les champs dans les tables de la base de données.
  - Les champs identifiants et de quelle manière ils sont gérés (séquence).
  - Les liaisons entre objets métier.

Copyright AEN-Intech 2020-2022

- Les annotations JPA sont généralement constituées de la façon suivante :

```
@Annotation(param1="Arthour", param2="Cuiller")
```

Parenthèses s'il y a des paramètres

Une virgule sépare les différents paramètres.

Les paramètres sont nommés avec un « = »

- Certaines annotations peuvent contenir d'autres annotations :

```
@JoinColumns({
        @JoinColumn(name="ADDR_ID", referencedColumnName="ID"),
        @JoinColumn(name="ADDR_ZIP", referencedColumnName="ZIP")
    })
```

Une virgule sépare les différentes sous-annotations.

Accolades pour spécifier des sous annotations (présentes même s'il n'y a qu'une seule sous-annotation).

Copyright AEN-Intech 2020-2022

- Les annotations JPA les plus utilisées sont les suivantes :

```java
@Entity
@Table(name = "Heros")
public class Heros {
 // Contenu de la classe
}
```

`@Entity` permet de spécifier une **classe** comme étant à persister
`@Table` permet de forcer le nom de la table en BDD

```java
@Id
@Column(name = "Id")
@GeneratedValue(strategy =GenerationType.IDENTITY)
public final int getId() {
     return this.id;
}
```

`@Id` permet de préciser que **ce getter** est celui de l'identifiant de l'entité
`@Column` permet de forcer le nom de la colonne en BDD
`@GeneratedValue` permet de préciser comment sont générés les identifiants (ici, grâce à une séquence interne à la BDD.

```java
@Column(name = "TotalXP", nullable = false)
public int getTotalXP() {
     return this.totalXP;
}
```

Sur un getter de type simple ou de type String, l'annotation **`@Column`** permet

- De spécifier que le champ correspondant est à persister ;
- de forcer le nom de la colonne en BDD ;
- De rajouter des contraintes à celui-ci

Copyright AEN-Intech 2020-2022

```java
@Enumerated(EnumType.STRING)
@Column(length = 14, name =
"PromotionLevel")
public final PromotionLevel getLevel() {
    return this.level;
}
```

Sur un getter de type **enum**, l'annotation **`@Enumerated`** permet de spécifier une le nom, la longueur et le type du champ dans BDD qui contiendra la valeur d'enum de l'entité.

- Si le paramètre est « **`EnumType.STRING`** », la valeur textuelle de l'enum sera sauvegardée dans un champ de type chaîne (option plus perenne)
- Si le paramètre est « **`EnumType.ORDINAL`** », la valeur entière de l'enum sera sauvegardée dans un champ de type int (option plus économe en place mais problématique si on rajoute une valeur au milieu de l'énum)

Notez que l'enum n'a pas besoin d'être une entité.

```java
@OneToOne
@JoinColumn(name = "PhysicalID")
public Physical getPhysical() {
    return this.physical;
}
```

`@OneToOne` permet de spécifier une relation 1→ 1 entre **2 entités** (donc, la classe **`Physical`** est forcement taguée **`@Entity`**). On le positionne au dessus **du getter**.
`@JoinColum` permet de spécifier le nom de la clé étrangère qui sera généré en BDD (et pointera sur **`Physical.Id`**).

Copyright AEN-Intech 2020-2022

```java
@OneToMany(fetch = FetchType.EAGER,
           cascade = CascadeType.ALL,
           orphanRemoval = true)
@JoinColumn(name = "fk_serie",
           referencedColumnName = "id")
public List<Weapon> getWeapons() {
    return this.weapons;
}
```

@oneToMany permet de spécifier une relation 1→n entre **2 entités** (donc, la classe **Weapon** est forcement taguée **@Entity**). On le positionne au dessus **du getter** d'une liste, d'une collection, etc.

- Le paramètre **fetch** permet de spécifier si la liste est remplie quand on lit l'entité (**FetchType.EAGER**) ou pas (**FetchType.LAZY**).
- Le paramètre **orphanRemoval** permet de spécifier si les enfants sont effacés si on efface le parent.

@JoinColum permet de spécifier le nom de la clé étrangère qui sera générée dans la table correspondant à l'entité **Weapon** (et pointera sur l'id de l'entité courante).

```java
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "HomeWorldID")
public HomeWorld getHomeWorld() {
    return this.homeWorld;
}
```

@ManyToOne permet de spécifier une relation n→ 1 entre **2 entités** (donc, la classe **HomeWorld** est forcement taguée **@Entity**). On le positionne au dessus **du getter**.
Le paramètre fetch permet de spécifier si l'extrémité « 1 » de la relation est ramenée quand on lit l'extrémité n (**FetchType.EAGER**) ou pas (**FetchType.LAZY**).
@JoinColum permet de spécifier le nom de la clé étrangère qui sera généré en BDD (et pointera sur **HomeWorld.Id**).

Copyright AEN-Intech 2020-2022

```java
@ManyToMany(fetch = FetchType.LAZY, cascade = CascadeType.ALL)
@JoinTable(name = "AlternativeEquipmentsList",
          joinColumns = @JoinColumn(name = "AlternativeEquipmentId"),
          inverseJoinColumns = @JoinColumn(name = "OwnedEquipmentsId")
)
public List<OwnedEquipments> getEquipmentsToChoose() {
    return this.equipmentsToChoose;
}
```

@ManyToMany permet de spécifier une relation n→ n entre **2 entités** (donc, la classe **OwnedEquipment** est forcement taguée **@Entity**). On le positionne au dessus **du getter** d'une liste, d'une collection, etc.

- Le paramètre **fetch** permet de spécifier si l'autre extrémité de la relation est ramenée quand on lit l'extrémité courante (**FetchType.EAGER**) ou pas (**FetchType.LAZY**) ;
- Le paramètre **cascade** permet de spécifier si les modifications sur l'entité courante sont reportées sur l'entité liée.

@JoinTable permet de spécifier le nom de la table de liaison qui servira à gérer la relation en BDD.

- Le paramètre **name** permet de spécifier le nom de la table de liaison en BDD ;
- Le paramètre **joinColumns** permet de spécifier la colonne qui contiendra la valeur d'identifiant de l'entité courante ;
- Le paramètre **inverseJoinColumns** permet de spécifier la colonne clé étrangère qui contiendra la valeur d'identifiant de l'entité **OwnedEquipment**).

Pour les annotations moins courantes, voir
https://www.objectdb.com/api/java/jpa/annotations

Copyright AEN-Intech 2020-2022



- Transformons les classes de notre package « **model** » en entités JPA en ajoutant un constructeur sans paramètre et en les annotant.
- Il ne faudra pas oublier de rajouter les entités définies au fichier persistence.xml afin qu'elles soient prises en compte.

```
<provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
<class>fr.pau.univ.series.model.Serie</class>
<class>fr.pau.univ.series.model.Saison</class>
<class>fr.pau.univ.series.model.Episode</class>
<exclude-unlisted-classes>true</exclude-unlisted-classes>
```

- Afin de tester la cohérence des annotations, faire un clic droit sur le projet puis « JPA Tools→Generate Tables from Entities »

Copyright AEN-Intech 2020-2022

- Ainsi, par exemple, l'annotation suivante dans l'entité « Serie »...

```
@OneToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "fk_serie", referencedColumnName = "id")
public List<Saison> getSaisons() {
    return this.saisons;
}
```

- ...va générer le code SQL suivant pour gérer la liaison entre les entités en BDD :

```
CREATE TABLE Saison (
    id INTEGER AUTO_INCREMENT NOT NULL,
    numero INTEGER,
    fk_serie INTEGER
    PRIMARY KEY (id)
);
ALTER TABLE Saison ADD CONSTRAINT FK_Saison_fk_serie
    FOREIGN KEY (fk_serie) REFERENCES Serie (id);
```

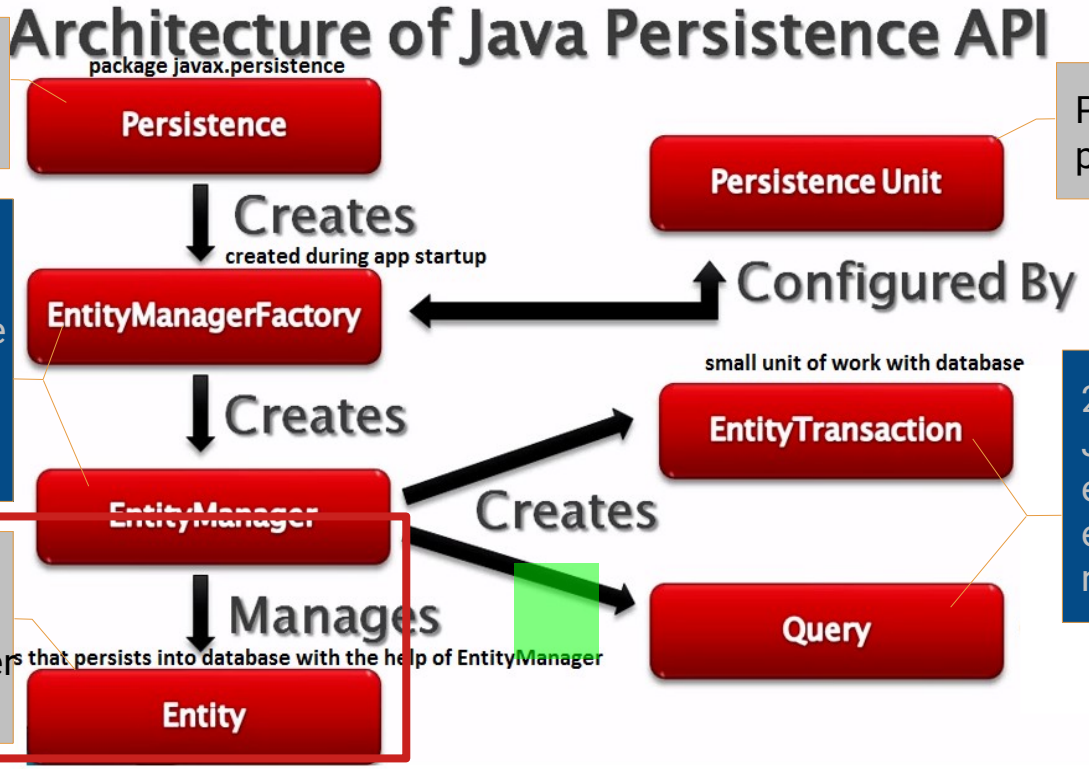
Copyright AEN-Intech 2020-2022

- Nous avons maintenant fini de définir les entités.

Ajout d'une implémentation JPA au projet.

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.



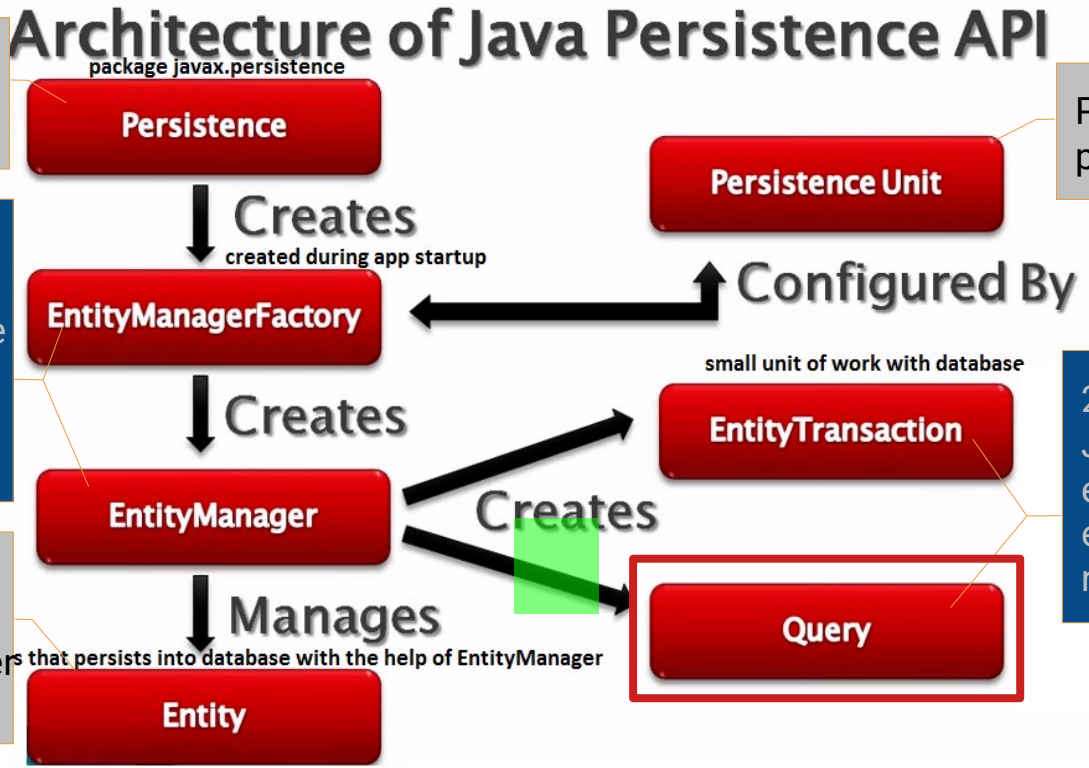
Paramétrage via le fichier persistence.xml

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Copyright AEN-Intech 2020-2022

- Voyons comment gérer les accès aux données en commençant par la lecture.

Ajout d'une implémentation JPA au projet.



Paramétrage via le fichier persistence.xml

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.

Copyright AEN-Intech 2020-2022

- Les requêtes, en JPA, sont définies via les annotations **`@NamedQueries`** et **`@NamedQuery`** que l'on positionne, en général, sous l'annotation **`@Entity`** qui les concernent :

```
@Entity
@Table(name = "Saison")
@NamedQueries({
    @NamedQuery(name="Saison.findById",
        query="SELECT s FROM Saison s WHERE s.id = :id"),
    @NamedQuery(name = "Saison.findAll",
        query = "SELECT s FROM Saison s"),
    @NamedQuery(name = "Saison.findByEpisode",
        query = "SELECT s FROM Saison s WHERE :id = ANY (SELECT e.id FROM s.episodes e)"),
    @NamedQuery(name = "Saison.findBySerie",
        query = "SELECT sais FROM Serie ser, IN(ser.saisons) sais WHERE ser.id = :id")
})
```

Comme son nom l'indique, une named query doit avoir un nom. Celui-ci doit être unique pour toute l'application. On le préfixe donc par le nom de l'entité qu'il renvoie.

La requête est paramétrable et écrite en JPQL (sorte de SQL mais qui travaille sur le modèle objet plutôt que sur la base de données.

- Les paramètres des requêtes sont les mots précédés de « : »

Copyright AEN-Intech 2020-2022

- Une fois les **@NamedQueries** et **@NamedQuery** écrites pour chaque **@Entity**, utilisons-les dans les Dao. 1er exemple avec la requête sans paramètres de **SerieDaoBdd.readAllSeries()** :

On utilise l'objet **TypedQuery** que l'on paramètre avec le type attendu (ici, on attend des objets **Serie**).

```
@Override
public List<Serie> readAllSeries() throws DaoException {
    TypedQuery<Serie> query = this.bdd.getEm().createNamedQuery("Serie.findAll", Serie.class);
    return query.getResultList();
}
```

C'est l'**EntityManager** stocké dans le singleton **DaoBddHelper** qui est chargé de construire les requêtes.

Le premier paramètre doit être le même que l'attribut **name** de l'une des « typed queries » définies dans les entités

Le 2e paramètre donne la classe attendue.Il doit être cohérent avec le typage du **TypedQuery**

Copyright AEN-Intech 2020-2022

- Le principe est le même pour les requêtes avec paramètres. 2e exemple avec la requête **`SerieDaoBdd.readSerie()`** :

```
@Override
public Serie readSerie(final int serieId) throws DaoException {
     final TypedQuery<Serie> query = this.bdd.getEm().createNamedQuery("Serie.findById", Serie.class);
     query.setParameter("id", serieId);
     final List<Serie> ret = query.getResultList();

     if (ret.size() > 0) {
          return ret.get(0);
     }
     return null;
}
```

Le premier paramètre doit être l'un des paramètres précédé d'un « : » dans la « typed query » définie dans l'entité. Le second peut être de n'importe quel type.

On teste la taille de la liste pour éviter une **`ArrayOutOfBoundException`**.

On renvoie **`null`** si on a rien trouvé.

- Remplir toutes les méthodes **`readXXX`** laissées auparavant vides résulte du même principe.

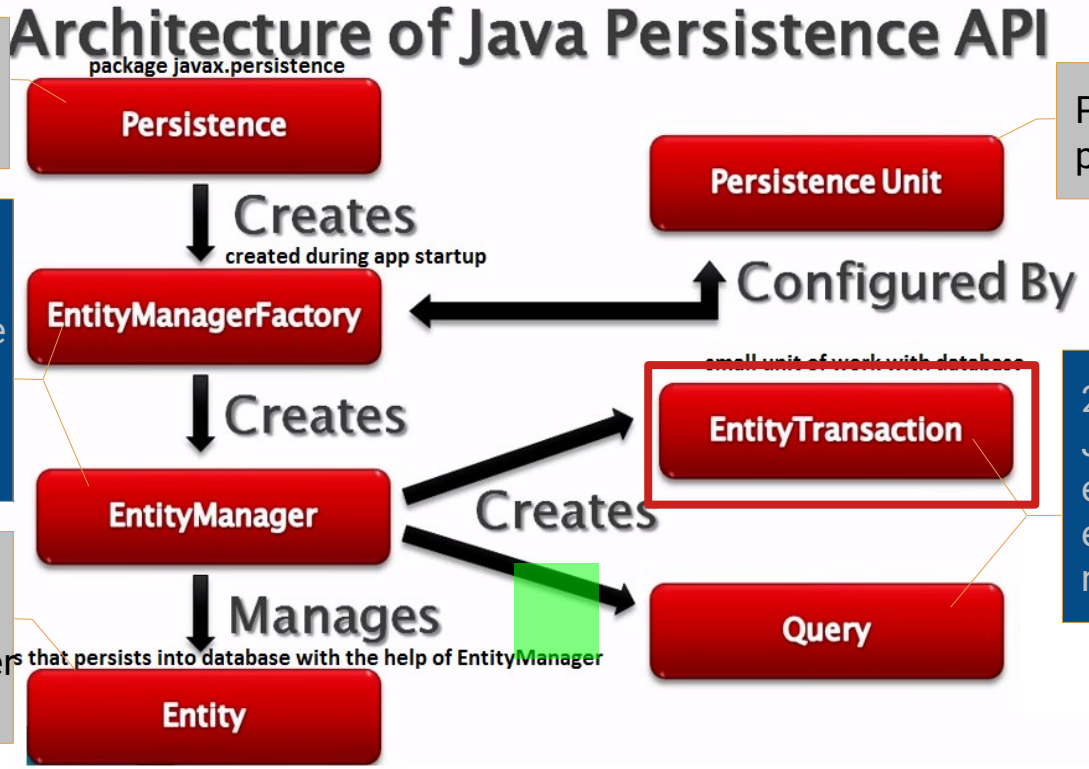
Copyright AEN-Intech 2020-2022

- Voyons maintenant comment gérer les accès hors lecture.

Ajout d'une implémentation JPA au projet.

1) Au démarrage de l'application ou du serveur, Une factory est créée afin de permettre la fabrication d'un Entitmanager .pour chaque Persistence unit gérée.

Les entités qui doivent être persistées sont annotées dans le code afin de spécifier à JPA comment les gérer.



Paramétrage via le fichier persistence.xml

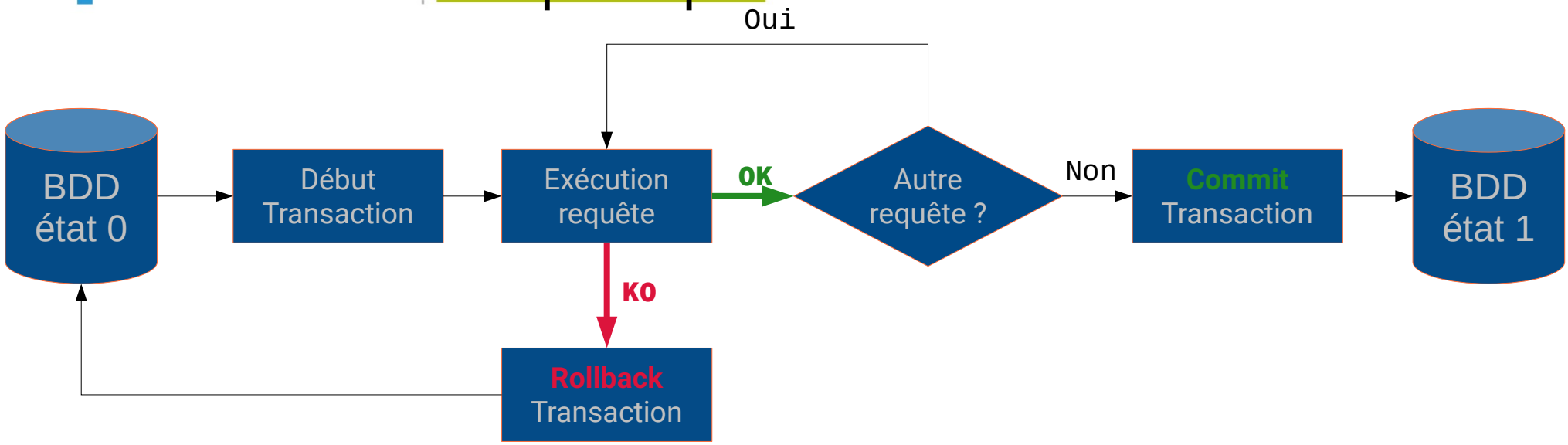
2) Pour accéder aux données, JPA utilise des requêtes écrites en JPQL pour la lecture et des transactions pour le reste.

Copyright AEN-Intech 2020-2022

- La transaction est une notion qui n'est pas propre à JPA mais concerne tout accès à une base de données. C'est un mécanisme de robustesse permettant de regrouper plusieurs requêtes (hors lecture) ensemble afin de :
  - Soit valider toutes les requetes en même temps si tout s'est bien passé ;
  - Soit, en cas d'erreur, annuler toutes les modifications en même temps.
- Exemple : annuler l'ajout d'une saison à une série si la création de la saison s'est mal passée.

Copyright AEN-Intech 2020-2022

- Voici un schéma de principe :



- La BDD n'est pas modifiée tant que le « Commit » n'a pas eu lieu. S'il y a la moindre erreur, la BDD retourne dans son état initial, en cas de succès, toutes les modifications sont appliquées.

Copyright AEN-Intech 2020-2022

- En JPA, c'est l'EntityManager qui est chargé de gérer les transactions. Codons les méthodes nécessaires dans la classe **`DaoBddHelper`**.

```
public void beginTransaction() {
     this.em.getTransaction().begin();
}

public void commitTransaction() {
     final EntityTransaction trans = this.em.getTransaction();
     if (trans.isActive()) {
          trans.commit();
     }
}

public void rollbackTransaction() {
     final EntityTransaction trans = this.em.getTransaction();
     if (trans.isActive()) {
          trans.rollback();
     }
}
```

On vérifie qu'il y a bien une transaction ouverte avant de faire le commit.

De même, on vérifie qu'il y a bien une transaction ouverte avant de faire le rollback.

Copyright AEN-Intech 2020-2022

- En JPA, Pour insérer une donnée (qui doit être annotée **`@Entity`**), il suffit d'appeler la méthode **`persist`** de l'**`EntityManager`**. Voici un exemple avec la création d'une série dans la classe **`SerieDaoBdd`** :

```
@Override
public Serie createSerie(final Serie s) throws DaoException {
    try {
        this.bdd.beginTransaction();
        this.bdd.getEm().persist(s);
        this.bdd.commitTransaction();
        return s;
    } catch (final PersistenceException e) {
        this.bdd.rollbackTransaction();
        throw new DaoException("Impossible de créer la série", e);
    }
}
```

Notez que la méthode renvoie maintenant une **`DaoException`**. Modifier l'interface en conséquence.

En cas d'erreur, on fait un rollback sur la transaction puis on encapsule l'Exception dans une **`DaoException`** pour la faire remonter.

- Remplir toutes les méthodes **`createXXX`** laissées auparavant vides résulte du même principe.

Copyright AEN-Intech 2020-2022

- En JPA, Pour modifier une donnée (qui doit être annotée **`@Entity`**), il suffit d'appeler la méthode **`merge`** de l'**`EntityManager`**. Voici un exemple avec la modification d'une série dans la classe **`SerieDaoBdd`** :

```java
@Override
public void updateSerie(final Serie s) throws DaoException {
    try {
        this.bdd.beginTransaction();
        this.bdd.getEm().merge(s);
        this.bdd.commitTransaction();
    } catch (final PersistenceException e) {
        this.bdd.rollbackTransaction();
        throw new DaoException("Impossible de modifier la série", e);
    }
}
```

Renvoie maintenant une **`DaoException`**. Modifier l'interface en conséquence.

En cas d'erreur, on fait un rollback sur la transaction puis on encapsule l'Exception dans une **`DaoException`** pour la faire remonter.

- Remplir toutes les méthodes **`updateXXX`** laissées auparavant vides résulte du même principe.

Copyright AEN-Intech 2020-2022

- En JPA, Pour supprimer une donnée (qui doit être annotée **`@Entity`**), il suffit d'appeler la méthode **`remove`** de l'**`EntityManager`**. Voici un exemple avec la supression d'une série dans la classe **`SerieDaoBdd`** :

```java
@Override
public void deleteSerie(final Serie s) throws DaoException {
    try {
        this.bdd.beginTransaction();
        this.bdd.getEm().remove(s);
        this.bdd.commitTransaction();
    } catch (final PersistenceException e) {
        this.bdd.rollbackTransaction();
        throw new DaoException("Impossible de supprimer la série", e);
    }
}
```

Renvoie maintenant une **`DaoException`**. Modifier l'interface en conséquence.

En cas d'erreur, on fait un rollback sur la transaction puis on encapsule l'Exception dans une **`DaoException`** pour la faire remonter.

- Remplir toutes les méthodes **`deleteXXX`** laissées auparavant vides résulte du même principe.

Copyright AEN-Intech 2020-2022

- Ajout d’une saison à une série :
  - Création de la saison puis MàJ de la série avec la nouvelle saison.
- Suppression d’une série :
  - Il faut d’abord supprimer toutes les saisons associées.
- Suppression d’une saison :
  - Il faut d’abord supprimer tous ses épisodes puis MàJ la série pour lui enlever cette saison.
- Ajout d’un épisode à une saison :
  - Création de l’épisode puis MàJ de la saison avec le nouvel épisode.
- Suppression d’un épisode...

Copyright AEN-Intech 2020-2022

- Toutes les méthodes des implémentations DAO doivent maintenant être codées. Mais nos transactions sont pour l'instant limitées à un seul accès BDD. On ne peux pas, tel que notre code est écrit, chaîner plusieurs acces base dans une seule transaction. Pourtant, si on essaie, par exemple, d'ajouter une saison à une série alors que la création de cette première n'a pas fonctionné correctement, cela ne se passera pas correctement.
- Nous allons donc rendre plus souple l'utilisation des transactions...

Copyright AEN-Intech 2020-2022



- En rajoutant un booléen à la signature de toutes les méthodes CUD (toutes hors lecture), on rend le système plus souple. Exemple avec la méthode **`deleteSerie`** :

```java
@Override
public void deleteSerie(final Serie s, final boolean useTransaction) throws
DaoException {
    try {
        if (useTransaction) {
            this.bdd.beginTransaction();
        }
        this.bdd.getEm().remove(s);
        if (useTransaction) {
            this.bdd.commitTransaction();
        }
    } catch (final PersistenceException e) {
        if (useTransaction) {
            this.bdd.rollbackTransaction();
        }
        throw new DaoException("Impossible de supprimer la série", e);
    }
}
```

Encore une modification de l'interface.

Toutes les méthodes servant à gérer les transactions sont encapsulées dans des tests du booléen.

Copyright AEN-Intech 2020-2022

- Par défaut, ce booléen sera à **`true`** mais quand on voudra chaîner plusieurs actions dans la même transaction...

```java
@WebServlet(urlPatterns = "/creerEpisode")
public class CreerEpisode extends HttpServlet {
    @Override
    protected void doPost(final HttpServletRequest request, final HttpServletResponse response)
            throws ServletException, IOException {
        try {
            int idSaison = Integer.parseInt(request.getParameter("saison"));
            String nomEpisode = request.getParameter("nomEpisode");
            if (nomEpisode != null && !nomEpisode.isBlank()) {
                Episode ep = new Episode(nomEpisode);
                Saison sais = daoSais.readSaison(idSaison);
                DaoBddHelper.getInstance().beginTransaction();
                ep = DaoFactory.getInstance().getEpisodeDao().createEpisde(ep, false);
                sais.addEpisode(ep);
                DaoFactory.getInstance().getSaisonDao().updateSaison(sais, false);
                DaoBddHelper.getInstance().commitTransaction();
            }
        } catch (DaoException e) {
            DaoBddHelper.getInstance().rollbackTransaction();
            request.setAttribute("erreur", e.getMessage());
        }
    }
}
```

Début transaction

Création épisode sans transaction.

MàJ saison sans transaction.

Commit transaction

Copyright AEN-Intech 2020-2022

- Toutes nos méthodes Dao hors lecture ont à peu près la même forme. On peut factoriser en créant une méthode dans la classe **`DaoBddHelper`** :

```
public void makePersistencyAction(Object obj, boolean useTrans, PersistencyAction action) throws PersistenceException {
     try {
          if (useTrans) {
               beginTransaction();
          }
          switch (action) {
               case PERSIST:
                    this.em.persist(obj);
                    break;
               case MERGE:
                    this.em.merge(obj);
                    break;
               case REMOVE:
                    this.em.remove(obj);
                    break;
          }
          if (useTrans) {
               commitTransaction();
          }
     } catch (final PersistenceException e) {
          if (useTrans) {
               rollbackTransaction();
          }
          throw e;
     }
}
```

Les 3 valeurs de l'enum **`PersistencyAction`**

Copyright AEN-Intech 2020-2022